DECLARATION

I hereby declare that this mini project report entitled "AI Forensic Insider Risk Review System" is a record of the project work carried out by me and submitted to Mar Athanasius College of Engineering, Kothamangalam, as part of the Master of Computer Applications programme, under the guidance of Dr. Manu John, Department of Computer Applications.

I further declare that this work has not formed the basis for the award of any degree, diploma, or other similar title to any candidate of any university or institution. The information and findings presented in this report are true to the best of my knowledge and belief.

Place: Kothamangalam

Date:

Signature of the Student

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my mini project guide, Dr. Manu John, for his valuable guidance, suggestions, and encouragement throughout the development of this project and the preparation of this report.

I extend my heartfelt thanks to Dr. Sonia Abraham, Head of the Department of Computer Applications, for her support and encouragement. I also thank the faculty members and staff of the Department of Computer Applications, Mar Athanasius College of Engineering, Kothamangalam, for their assistance.

I acknowledge the creators of the CERT Insider Threat Test Dataset for making the synthetic data available for research and education. I am also grateful to the developers and maintainers of Python, pandas, NumPy, scikit-learn, FastAPI, React, Vite, and Streamlit, whose tools supported the implementation of this project.

Finally, I express my heartfelt gratitude to my parents, family, and friends for their encouragement, patience, and support. I thank everyone who contributed directly or indirectly to the completion of this project and report.

- Hrishikesh P S

ABSTRACT

This mini project presents a prototype for reviewing employee activity records for patterns associated with known insider-threat scenarios. It processes the synthetic CERT r4.2 logon, device, file, email, and web activity data, cleans the records, and aggregates them into daily employee-level features. Incident-window labels from the CERT answer file are used to compare Logistic Regression, Decision Tree, Random Forest, and Isolation Forest models. Employees are kept separate across training, validation, and test sets.

The validation-selected Random Forest achieves an F1 score of 0.200 on the held-out test set at the selected one-percent review capacity, with 14.35% precision and 32.87% recall. These results show that the model can help rank synthetic employee-days for review, but incomplete labels and low precision mean that a score is not evidence of wrongdoing.

A local dashboard built with React and FastAPI lets an analyst select activity CSV files and inspect ranked review leads and their supporting activity counts. The prototype demonstrates data processing, model evaluation, and local deployment while emphasizing that any conclusions require authorized, accountable human review and corroborating evidence.

LIST OF TABLES

3.1 Activity source inventory 4
3.2 Data-cleaning summary from saved quality output 4
3.3 Employee-day feature definitions 5
3.4 Functional requirements and implementation evidence 11
3.5 Non-functional requirements and current status 12

5.1 Test cases and verification status 18
5.2 Metrics at the 1% queue rate selected by validation F1 19
5.3 Model evaluation from saved `model_comparison.csv` 19
5.4 Additional test metrics from saved evaluation 19
5.5 Random Forest confusion matrix at the validation-selected 1% queue rate . . . 21
5.6 Requirements traceability 21

7.1 Selected repository history 23

10.1 Confusion-matrix counts at the validation-selected 1% queue rate 27
10.2 Offline evaluation and interactive scoring boundaries 56
10.3 Input columns and derived values in the dashboard processor 56
10.4 Supplementary acceptance and quality plan 59
10.5 Suggested run-record fields for a future demonstration 61

LIST OF FIGURES

3.1 Total retained events by source in the saved analysis notebook 6
3.2 Daily recorded activity volume across the dataset period 6
3.3 Ten synthetic users with the highest aggregate event counts 6
3.4 Random Forest training, selection, and final evaluation flow 8
3.5 Isolation Forest anomaly-scoring flow 9
3.6 High-level project pipeline 9

4.1 Use-case model for the prototype 14
4.2 Activity flow for one dashboard review 14
4.3 Sequence of an analysis request 15
4.4 Component design and data flow of the project 15
4.5 Landing screen with folder selection and the review guide collapsed. 17
4.6 Expanded scoring guide with model descriptions and the review rule. 17

CONTENTS

1 Introduction **1**

1.1 Background . . . 1
1.2 Problem statement . . . 1
1.3 Objectives of the Project . . . 1
1.4 Scope of the Project . . . 1
1.5 Project Domain and Technology Stack . . . 2
1.6 Organisation of the Report . . . 2

2 Supporting Literature **3**

2.1 Literature Review . . . 3
2.2 Literature Review Summary . . . 3
2.3 Findings and Proposals . . . 3

3 System Analysis **4**

3.1 Analysis of the dataset . . . 4
3.1.1 About the dataset . . . 4
3.1.2 Explore the dataset . . . 4
3.2 Data preprocessing . . . 5
3.2.1 Data cleaning . . . 5
3.2.2 Analysis of feature variables . . . 5
3.2.3 Analysis of class variables . . . 5
3.3 Data visualization . . . 5
3.4 Analysis of architecture and algorithms . . . 7
3.4.1 Detailed study of proposed algorithms . . . 7
3.4.2 Step-by-step explanation of model operation . . . 7
3.4.3 Algorithm flow diagrams . . . 8
3.5 Project pipeline . . . 8
3.5.1 Explanation of pipeline stages . . . 8
3.6 Feasibility analysis . . . 10
3.6.1 Technical feasibility . . . 10
3.6.2 Economic feasibility . . . 10
3.6.3 Operational feasibility . . . 10
3.7 System environment . . . 10
3.7.1 Software environment . . . 10
3.7.2 Hardware environment . . . 10
3.8 Actors and roles . . . 11
3.9 Functional and non-functional requirements . . . 11

4 System Design **13**
4.1 Model(s) Building . . . 13
4.1.1 Implementation code . . . 13
4.1.2 Model planning and training/testing split . . . 13
4.1.3 Training . . . 13
4.1.4 Testing . . . 13
4.2 System Design Diagrams . . . 14
4.2.1 Use-case model . . . 14
4.2.2 Activity diagram . . . 14
4.2.3 Sequence diagram . . . 15
4.2.4 Class and component design . . . 15
4.3 Project pipeline and output design . . . 15
4.4 Database design . . . 16
4.5 UI design . . . 16

5 Results and Discussion **18**
5.1 Testing approach . . . 18
5.2 Model planning and evaluation protocol . . . 18
5.3 Comparison of models . . . 19
5.4 Evaluation metrics . . . 20
5.5 Confusion matrix . . . 20
5.6 Discussion . . . 21
5.7 Requirements traceability . . . 21

6 Model Deployment **22**
6.1 Local demonstration setup . . . 22
6.2 Analyst workflow and limitations . . . 22

7 Git History **23**

8 Conclusions **24**
8.1 Learning Outcomes . . . 24
8.2 Technical Skills Acquired . . . 24
8.3 Analytical and Professional Learning . . . 24
8.4 Summary of the Project . . . 24

9 Future Scope **25**

10 Appendix **26**
10.1 Minimum Software and Hardware Requirements . . . 26
10.1.1 Minimum software requirements . . . 26
10.1.2 Minimum hardware requirements . . . 26

10.2 Output Artifact Guide . . . 26
10.3 Selected Test Counts . . . 26
10.4 Responsible Use Checklist . . . 27
10.5 Selected Code Snippets . . . 27
10.6 Complete Source Listings . . . 29
10.6.1 Analysis notebook source . . . 29
10.6.2 FastAPI service source . . . 44
10.6.3 Feature processing and ranking source . . . 47
10.6.4 Alternative Streamlit interface source . . . 51
10.7 Supplementary Project Record . . . 55
10.7.1 Two Distinct Project Workflows . . . 55
10.7.1.1 Offline development workflow . . . 55
10.7.1.2 Interactive scoring workflow . . . 56
10.7.2 Detailed Input Contract . . . 56
10.7.2.1 Feature alignment . . . 57
10.7.3 Review-Queue Calculation . . . 57
10.7.3.1 Signal text and its limitations . . . 58
10.7.4 Illustrative Review Walkthrough . . . 58
10.7.5 Proposed Acceptance and Quality Plan . . . 59
10.7.6 Artifact and Run Record . . . 60
10.7.7 Known Technical Boundaries . . . 60

References **62**

INTRODUCTION

1.1 Background

Organisations record routine digital activity in access, device, communication, file, and web logs. Those records can help an investigator understand when an account or device behaved unusually. The volume and variety of logs make manual review difficult, so a data-processing and ranking tool can help prioritise records for inspection.

This project is an educational demonstration of that idea. It converts the CERT r4.2 synthetic activity data into daily aggregates, compares several machine-learning approaches against known incident windows, and provides a local dashboard to explore high-priority employee-days. A model score is a prioritisation signal only. It does not establish intent, identity, policy violation, or guilt.

1.2 Problem statement

The prototype needs to turn a large collection of heterogeneous activity CSV files into a compact and inspectable review queue. The analysis must handle missing or malformed records, combine sources at a consistent employee-day granularity, evaluate models under severe class imbalance, and show investigators what activity contributed to a rank. Since only known incidents are labelled, a record outside a listed incident window must not be described as definitively normal.

1.3 Objectives of the Project

Aim: Build and evaluate a local, explainable-at-the-signal-level prototype that ranks employee activity records for human review.

Objectives

1. Inspect and clean five CERT r4.2 activity CSV sources.
2. Aggregate activity into daily features for each user.
3. Create a binary known-incident-window label from the answer file.
4. Compare three supervised classifiers and one unsupervised anomaly detector using employee-grouped data splits.
5. Evaluate count-aware synthetic augmentation using training data only, with variant selection on validation Average Precision.
6. Evaluate ranking quality at fixed review capacities, selecting the reported one-percent operating point using validation F1 and retaining five percent as a comparison.
7. Save model artifacts and expose a local React dashboard with a FastAPI service for human review of new activity records.

1.4 Scope of the Project

The implemented feature set contains daily event counts, after-hours counts, device connection counts, and email attachment totals when those fields exist. It does not analyse message bodies, web URLs, or file contents. The model target is whether an employee-day overlaps a listed

incident window in the r4.2 answer file. The prototype is not a live monitoring system, identity-resolution system, case-management platform, or automated disciplinary tool. The primary demonstration interface is a local React dashboard backed by FastAPI. The earlier Streamlit interface remains available as an alternative.

1.5 Project Domain and Technology Stack

The project falls within applied data analysis, supervised classification, and anomaly detection. Python is used for feature preparation and model development. pandas and NumPy support tabular processing; scikit-learn provides the estimators, grouped splitting, and evaluation metrics; joblib stores the fitted models; React and FastAPI provide the local interactive demonstration [8, 9]; and Streamlit remains available as an alternative. Jupyter Notebook is used for the reproducible analysis.

1.6 Organisation of the Report

Chapter 2 reviews supporting literature and project proposals. Chapter 3 presents system analysis, including the dataset, preprocessing, algorithms, feasibility, and environment. Chapter 4 describes model building and system design. Chapter 5 reports evaluation results and discussion; Chapter 6 documents the local deployment. Chapter 7 records selected Git history and project challenges, followed by learning outcomes, conclusion, future work, references, and appendices.

SUPPORTING LITERATURE

2.1 Literature Review

Insider-threat detection is challenging because legitimate and harmful activity can look similar in system logs. A late login, removable-media connection, or large attachment is a review signal only when considered with context; no single event establishes intent. The CERT Insider Threat Test Dataset supports reproducible study through synthetic activity and scenario labels, but it cannot establish performance in a real organisation [1].

This project compares three supervised methods with one unsupervised method. Logistic Regression provides a linear baseline; a Decision Tree models feature interactions through splits; and Random Forest combines randomized trees to improve stability on tabular data [2]. Isolation Forest ranks observations that are easier to isolate through random splits, without using incident labels during fitting [3]. Its anomaly score represents unusualness in the selected features, not incident probability.

Because known positive days are rare, accuracy alone can conceal poor retrieval. Precision describes the share of reviewed records matching a known label, while recall measures the share of known labels retrieved. Average Precision evaluates ranking across score cutoffs and is useful for this imbalanced task [4]. The experiment therefore reports ranking metrics and fixed review-capacity results, selecting model and capacity on validation data before evaluating the held-out test employees.

2.2 Literature Review Summary

The methods answer different questions: supervised models learn the patterns present in known labels, while Isolation Forest ranks feature space deviations. Label quality, feature design, employee separation, and review capacity limit the conclusions for every method.

2.3 Findings and Proposals

- Report precision, recall, and ranking quality alongside accuracy.
- Separate employees across training, validation, and test sets; select models on validation data and reserve test data for final reporting.
- Treat unmatched labels as unknown and require contextual human review before acting on a ranked lead.

SYSTEM ANALYSIS

3.1 Analysis of the dataset

3.1.1 About the dataset

The experiment reads the CERT r4.2 release from the local `r4.2` folder and the matching `answers/insiders.csv` answer file. The activity inputs are listed in Table 3.1. The answer file is used only for research labels and model evaluation/training; the dashboard's user-selected activity folder does not need the answer file.

Table 3.1: Activity source inventory

Source	Fields used by the first-stage pipeline
`logon.csv`	Timestamp, user, activity records
`device.csv`	Timestamp, user, activity; connection events counted
`file.csv`	Timestamp, user, file activity records
`email.csv`	Timestamp, user, attachment count
`http.csv`	Timestamp, user, web activity records
`answers/insiders.csv`	Dataset version, user, incident start/end, scenario

The saved labelled table contains 330,452 employee-days for 1,000 synthetic user IDs, covering 2 January 2010 through 17 May 2011. It contains 1,364 positive employee-days (0.413%) under the notebook's labeling rule. The saved test partition has 66,118 employee-days and 289 positive labels. These are reproducible facts from the project's saved artifacts, not general statistics about real organisations.

3.1.2 Explore the dataset

The five files contain approximately 32.77 million raw event rows. The cleaning summary reports 32,409,354 rows kept and 360,868 duplicates removed. No missing user IDs or invalid dates were reported in the saved quality summary. Deduplication is performed within each input chunk, so duplicates spanning separate chunks may remain; this is a limitation to verify if exact global deduplication is required.

Table 3.2: Data-cleaning summary from saved quality output

Source	Rows read	Rows kept	Duplicates removed
Logon	854,859	854,859	0
Device	405,380	405,374	6
File	445,581	445,581	0
Email	2,629,979	2,618,056	11,923
HTTP	28,434,423	28,085,484	348,939
Total	32,770,222	32,409,354	360,868

The positive class is highly sparse. The 1,364 positive days belong to 70 user IDs, and many users have no known positive days. A group-aware split therefore matters: random row splitting could place records from the same person in training and testing and produce an overly optimistic estimate for generalization to unseen employees.

3.2 Data preprocessing

3.2.1 Data cleaning

Each CSV is read in chunks of 25,000 rows to limit memory use. Column names and user IDs are normalized, timestamps are parsed, rows missing a usable user or date are excluded, and duplicate rows within a chunk are removed. Each retained record contributes to a calendar-day aggregate. Where source fields are available, device "connect" events and email attachment totals are separately accumulated. Missing source/day combinations are filled with zero after source tables are combined.

3.2.2 Analysis of feature variables

The modelling table contains the numeric activity features below. The model excludes user ID, date, scenario, and the target label.

Table 3.3: Employee-day feature definitions

Feature	Meaning
logon_event_count	Number of retained logon events that day
logon_after_hours_count	Logon events before 06:00 or at/after 18:00
device_event_count	Number of retained device events
device_after_hours_count	Device events in the defined after-hours interval
device_device_connect_count	Device activity records marked "connect"
file_event_count	Number of retained file events
file_after_hours_count	File events in the defined after-hours interval
email_event_count	Number of retained email events
email_after_hours_count	Email events in the defined after-hours interval
email_attachment_count	Sum of parsed email attachment counts
http_event_count	Number of retained web events
http_after_hours_count	Web events in the defined after-hours interval

The feature set intentionally excludes content, URLs, and filenames. This reduces sensitive content processing but also limits detection to coarse-volume and timing signals. The "after-hours" definition is a fixed clock interval (before 06:00 or at/after 18:00); it does not account for individual schedules, holidays, or time zones.

3.2.3 Analysis of class variables

For each answer-file incident in dataset version 4.2, the notebook normalizes start and end timestamps to calendar days and marks matching user-days from the start day through the end day, inclusive. A label of 1 therefore means "this employee-day overlaps a listed incident window." A label of 0 means only that it was not marked by this specific answer-file rule. It does not assert benign conduct. Day-level labeling can include normal activity on the same day as an incident, and can miss relevant activity outside its listed interval.

3.3 Data visualization

The notebook includes source event-count plots, employee activity summaries, label counts, a model comparison table, and confusion-matrix plots. The comparison visualizations should

be read alongside the positive-class support and both the selected one-percent capacity and five-percent reference, since majority-class accuracy can obscure incident-class performance.

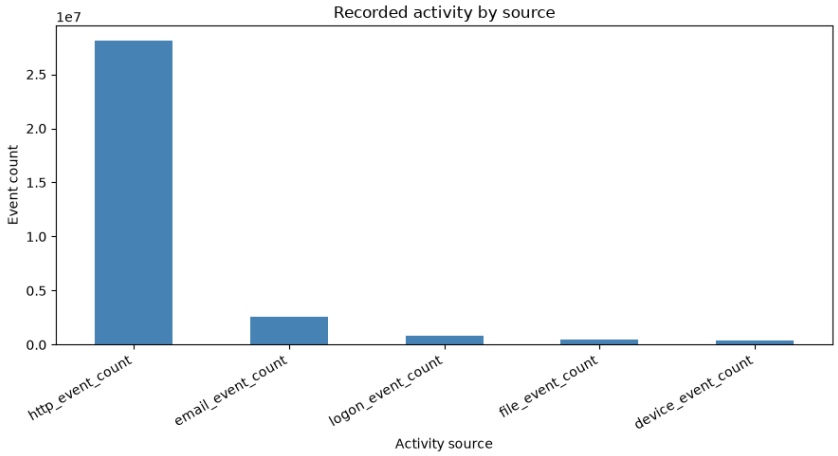


Figure 3.1: Total retained events by source in the saved analysis notebook

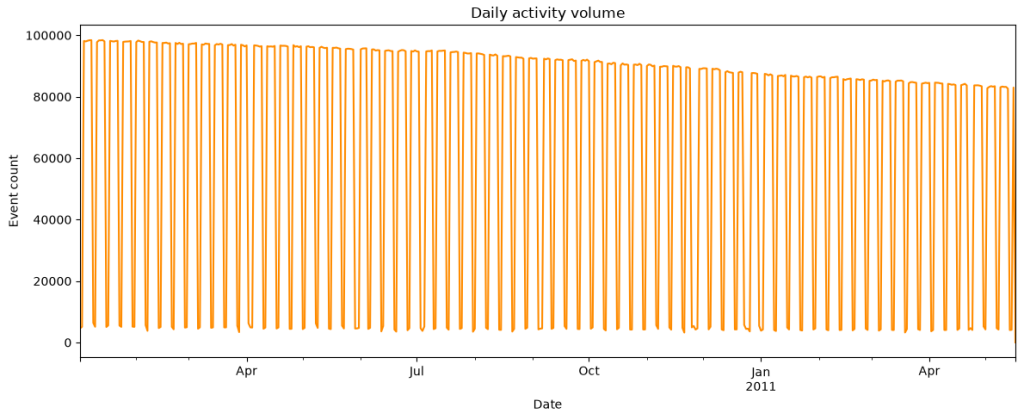


Figure 3.2: Daily recorded activity volume across the dataset period

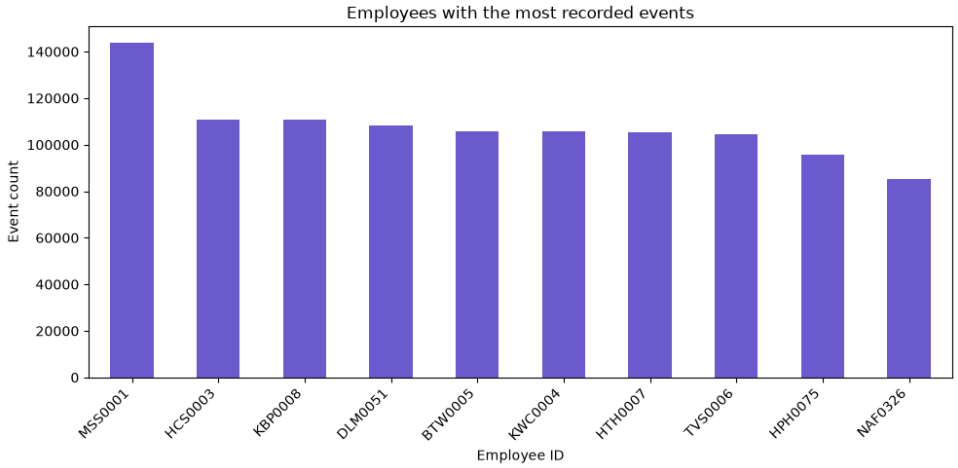


Figure 3.3: Ten synthetic users with the highest aggregate event counts

The first plot makes the scale imbalance among event sources visible: HTTP contributes most of the retained rows. The daily plot shows a recurring low-activity pattern alongside a declining baseline over time; it is a dataset overview, not evidence that any user is anomalous. The employee plot is descriptive only and does not adjust for working days, role, or tenure.

3.4 Analysis of architecture and algorithms

3.4.1 Detailed study of proposed algorithms

Logistic Regression: A supervised baseline with median imputation, standard scaling, class weighting, and a maximum of 1,000 iterations. The model is fitted on the training partition and outputs positive-class scores.

Decision Tree: A supervised tree with maximum depth 10, minimum leaf size 10, and balanced class weights. These constraints limit tree complexity compared with an unrestricted tree.

Random Forest: A supervised ensemble of 200 trees, minimum leaf size 5, balanced-subsample class weights, and parallel fitting. It averages tree predictions to produce a positive-class score.

Isolation Forest: An unsupervised anomaly detector with 200 trees, automatic contamination handling, and parallel fitting. The negative decision function is used so larger values rank as more unusual. The contamination setting controls the estimator's outlier decision threshold; the project separately evaluates a validation- selected one-percent queue and retains five percent as a reference.

3.4.2 Step-by-step explanation of model operation

Random Forest:

1. Prepare a numeric feature row for each employee-day and retain the matching known-window label for supervised fitting.
2. For the augmentation candidate, select positive training rows only and create count-perturbed variants using Poisson and binomial resampling. Validation and test rows are not augmented.
3. Fit 200 decision trees. Each tree receives a bootstrap sample; class weighting compensates for unequal label counts in that bootstrap sample. The configured minimum leaf size is five.
4. For an input row, collect the positive-class output from every tree and average the outputs to obtain the forest score.
5. Calculate validation Average Precision for the real-data and augmented candidates. Retain augmentation only when the validation result is higher than the baseline.
6. After the model choice is fixed, calculate test scores and the precision, recall, and F1 values at the validation-selected capacity. Do not change the model based on test results.

Isolation Forest:

1. Fit the estimator to the training feature matrix without incident labels.
2. Construct isolation trees through random feature and split choices; unusual observations tend to be separated with shorter paths.
3. Compute each observation's decision score and reverse its sign so the larger-is-higher convention matches the other model views.
4. Compare the resulting ranking with known-window labels only during validation and test

evaluation; labels do not influence the unsupervised fit.

3.4.3 Algorithm flow diagrams

The Random Forest workflow used by the project is summarized in Figure 3.4. Each estimator tree is trained from a bootstrap sample with balanced class weighting; the trees are combined to produce a supervised positive-class score. In the selected variant, the positive employee-day training rows are augmented using count-aware resampling before fitting. Validation data selects whether the augmented candidate is retained. The test partition is not used in that selection.

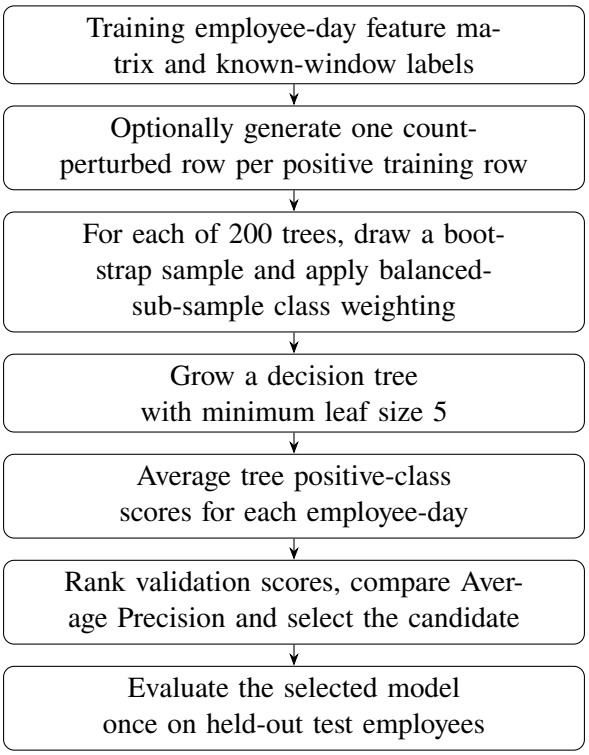


Figure 3.4: Random Forest training, selection, and final evaluation flow

Figure 3.5 shows the separate unsupervised anomaly path. It does not use the incident labels while fitting. The estimator isolates records using random splits, then the score direction is reversed so larger values appear higher in the common ranking interface.

3.5 Project pipeline

3.5.1 Explanation of pipeline stages

Source ingestion: The first stage identifies the five named activity files. The answer file is a separate research input used by the offline notebook; it is not required by the interactive scoring service. Separating these inputs keeps the dashboard scoring path from depending on evaluation labels.

Cleaning and aggregation: Each source is read in chunks. User identifiers and timestamps are normalized, unusable rows are discarded, and duplicates within each chunk are removed. Retained events are converted to user-day counts and source-specific totals. The outputs of the five processors are aligned on a common user-day key and combined into a numeric feature matrix.

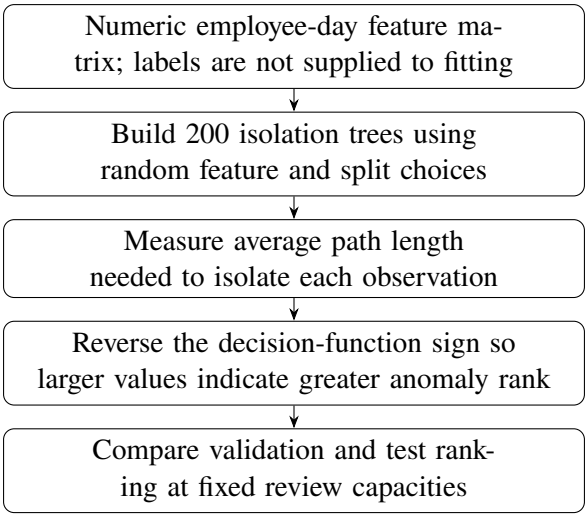


Figure 3.5: Isolation Forest anomaly-scoring flow

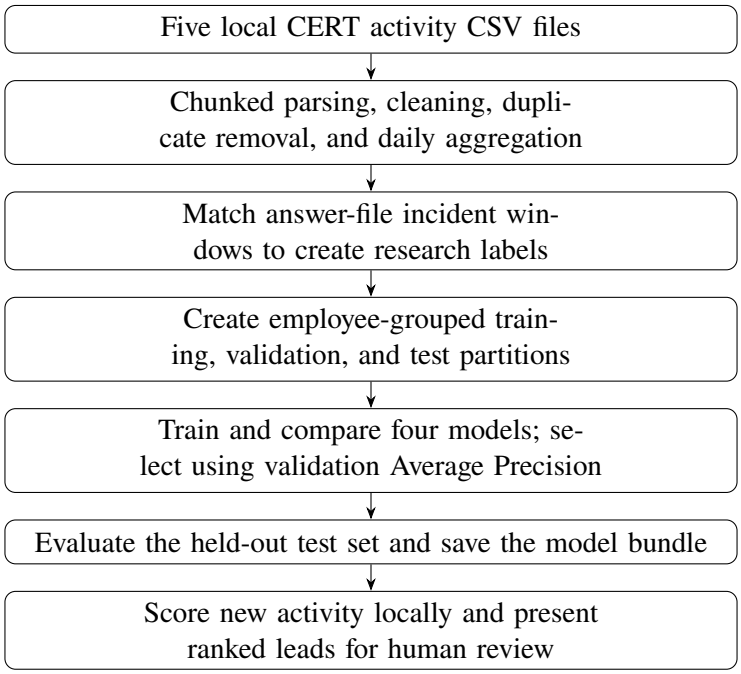


Figure 3.6: High-level project pipeline

Labeling and partitioning: In offline development, the answer file's inclusive incident dates are matched to the corresponding user and calendar day. The resulting target indicates a known-window match. Employee groups are then assigned to training, validation, and test partitions so that one user does not appear in more than one partition.

Model comparison: Candidate models are fitted on training records. Validation Average Precision is used for ranking comparison, and the review capacity is selected from the declared capacity grid using validation F1. The held-out test groups are used after these choices have been fixed. This order limits the use of the test set to final performance reporting.

Artifact and review: The final notebook step serializes the estimators and ordered feature columns. The API loads that bundle, recreates the feature matrix for a selected folder, computes model scores and percentile ranks, and writes the employee-level and employee-day CSV files. The React interface displays these outputs as review leads; an analyst must consult original

records and context outside the application.

3.6 Feasibility analysis

3.6.1 Technical feasibility

The analysis pipeline is implemented in Python using pandas, NumPy, scikit-learn, and joblib. FastAPI serves the local analysis service; the React client is built with Vite, and Streamlit remains an alternative interface. Chunked CSV processing makes the main event ingestion feasible on a developer workstation. The repository includes the dataset directory locally, while generated outputs are ignored by Git. The demonstrated environment is macOS on Apple silicon with Python 3.14.8, pandas 3.0.6, NumPy 2.5.3, scikit-learn 1.9.1, FastAPI, and React. These versions describe the checked environment and are not a claim of a fully reproducible environment lockfile.

3.6.2 Economic feasibility

The prototype uses free, open-source Python libraries and runs locally. The main costs are developer time, storage for the raw logs and derived outputs, and the compute time required to process large CSV files. No cloud service, paid API, or hosted database is required for the demonstration. Production costs would additionally include access controls, auditing, secure retention, model monitoring, and analyst review capacity.

3.6.3 Operational feasibility

The intended operator is an authorised analyst who can choose a folder of expected CSVs, run analysis, inspect a ranked queue, and compare model rank scores with underlying activity days. Operational use would require documented policies, access authorization, a defined alert capacity, case notes, independent corroboration, and a process to correct inaccurate data or interpretations. Those governance systems are not implemented in this prototype.

3.7 System environment

3.7.1 Software environment

- Operating system used for verification: macOS 27.0.1, Apple silicon.
- Python: 3.14.8.
- pandas 3.0.6; NumPy 2.5.3; scikit-learn 1.9.1.
- FastAPI and Uvicorn for the local API; React and Vite for the primary web client; Streamlit 1.65.0 for the alternative client. Joblib persists the estimator bundle.
- Jupyter notebook: `notebook/forensic.ipynb`.
- Primary demo entry points: `api.py` and `frontend/`; alternative entry point: `app.py`.

3.7.2 Hardware environment

The checked development environment is a Mac with Apple silicon; the repository does not record a verified RAM, storage, or processor-core specification. The program parallelizes selected estimator operations and processes large source files in chunks. Hardware recommen-

dations should be established by measuring end-to-end runtime and peak memory on the target deployment machine rather than inferred from this report.

3.8 Actors and roles

Analyst Selects the activity folder, runs analysis, reviews ranked users and their supporting activity-day indicators, and validates leads independently.

Project developer Maintains preprocessing, model training, evaluation, and dashboard code.

Supervisor / administrator Authorizes use, controls access to sensitive logs, and defines retention and review procedures.

Model Computes relative scores from the engineered numeric features; it does not make employment or disciplinary decisions.

3.9 Functional and non-functional requirements

The requirements below describe the demonstrated prototype, not a production insider-risk service. Requirements that concern governance and operational assurance are stated as constraints because the current repository does not implement them.

Table 3.4: Functional requirements and implementation evidence

ID	Requirement	Evidence / limitation
FR01	Accept the five named activity CSV files from a selected local folder	React folder picker and required-file validation in `api.py`
FR02	Read large files incrementally rather than loading each entire source at once	25,000-row pandas chunks
FR03	Exclude rows without a usable user or parsed date and remove duplicates within each chunk	Notebook and dashboard preprocessing; cross-chunk duplicates may remain
FR04	Aggregate retained records by user and calendar day into numeric activity features	Notebook and `process_activity_file()`
FR05	Align dashboard features to the ordered feature list used at training time	Saved model bundle; missing expected columns are filled with zero
FR06	Produce comparative model scores and a ranked employee review list	Four estimators and consensus ranking in `score_folder()`
FR07	Show signals and high-ranked activity days supporting a review lead	React interface; signal rules are descriptive, not causal explanations
FR08	Save review summaries as CSV artifacts	`dashboard_report.csv` and `dashboard_day_scores.csv`

Table 3.5: Non-functional requirements and current status

ID	Requirement	Current status
NFR01	Keep the demonstration local and avoid sending activity data to an external service	Implemented for the demonstrated flow
NFR02	Handle large event files within workstation memory constraints	Chunked input is implemented; peak memory has not been measured
NFR03	Make outputs understandable and subject to human review	Caution text and activity context are shown; no case-management workflow
NFR04	Protect raw records and generated outputs from unauthorized access	Not implemented; local permissions remain the operator's responsibility
NFR05	Record access, analyst decisions, model versions, and retention actions	Not implemented
NFR06	Demonstrate generalization to operational data and acceptable review burden	Not established by the synthetic, employee-day experiment

The requirements highlight the boundary between a working prototype and a deployable decision-support system. File ingestion and ranking are present, but access control, auditability, retention, and evidence management require separate design and validation before operational use.

SYSTEM DESIGN

4.1 Model(s) Building

The implementation is divided between the analysis notebook and the React/FastAPI application. The notebook performs bulk preparation, incident-window labelling, grouped evaluation, and model serialization. The application loads the saved feature list and estimator bundle, rebuilds employee-day features from selected CSV files, and displays ranked review leads.

4.1.1 Implementation code

The notebook is the source of the training and evaluation workflow. The primary dashboard uses `frontend/src/App.jsx` for the React interface, `api.py` for its FastAPI endpoints, and `forensic_core.py` for CSV parsing, feature alignment, four-model scoring, and employee ranking. The Streamlit alternative is implemented in `app.py`. Selected source excerpts are included in the appendix; the full source files remain in the project repository.

4.1.2 Model planning and training/testing split

The labelled employee-day records are separated by employee identifier, not by randomly splitting individual days. This keeps one synthetic employee's records within a single partition and reduces leakage from repeated observations of the same employee. The training partition is used to fit the estimators; the validation partition is used to select a model by Average Precision; the test partition is reserved for final comparison. Partition sizes and positive-label counts are reported in Chapter 5.

4.1.3 Training

Logistic Regression, Decision Tree, Random Forest, and Isolation Forest are fitted using the configured preprocessing and estimator settings described in the system-analysis chapter. The first three use the training labels; Isolation Forest is fitted as an unsupervised anomaly detector without those labels. The notebook stores the model objects, their names, and ordered feature columns in a joblib bundle for the dashboard.

4.1.4 Testing

The held-out employee groups are scored after training. Supervised scores use the positive-class output; Isolation Forest scores are direction-adjusted so larger values correspond to more unusual observations. Model selection and the test metrics are kept distinct. The saved comparison and confusion-matrix counts are detailed in Chapter 5; the test-set results do not validate the dashboard's separate employee-level consensus rule.

4.2 System Design Diagrams

4.2.1 Use-case model

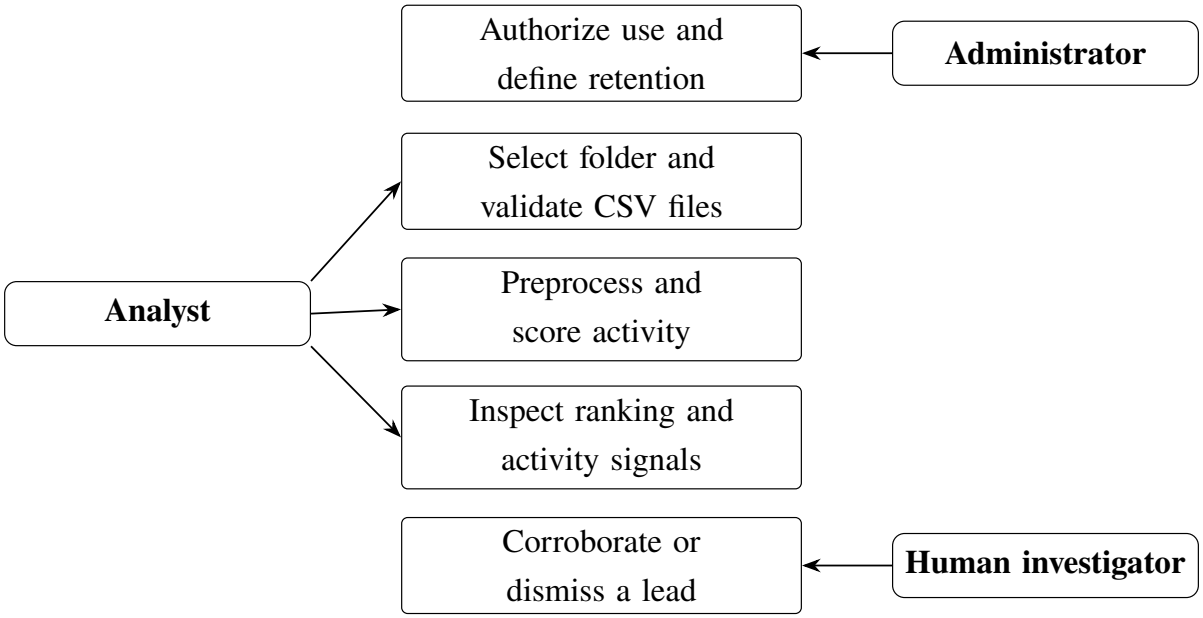


Figure 4.1: Use-case model for the prototype

4.2.2 Activity diagram

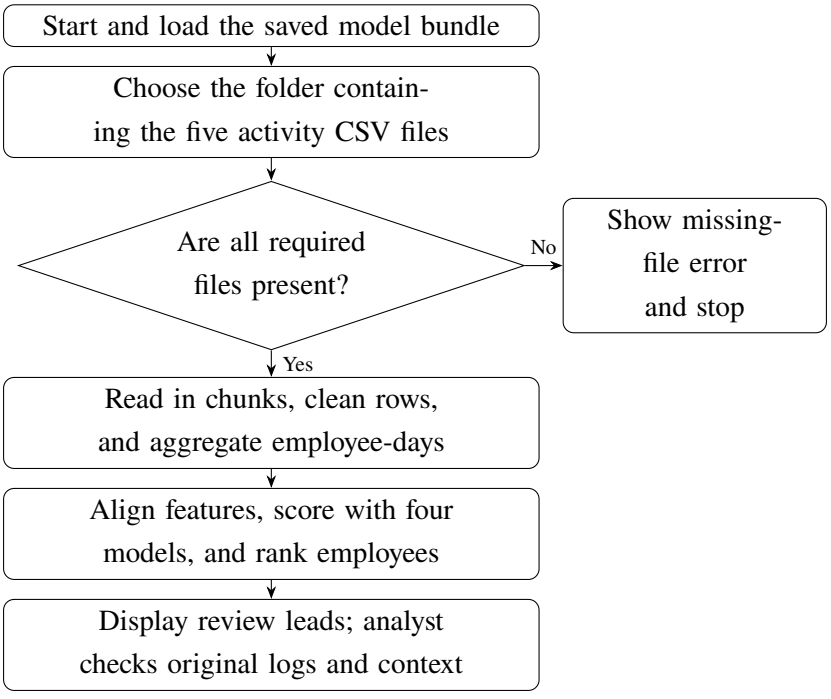


Figure 4.2: Activity flow for one dashboard review

4.2.3 Sequence diagram

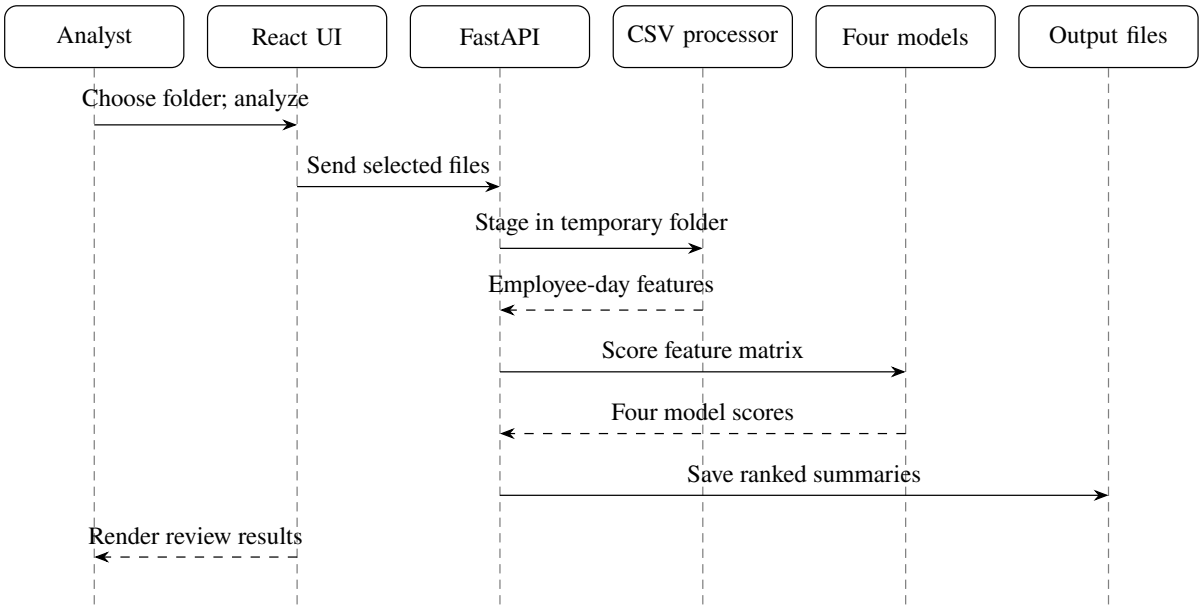


Figure 4.3: Sequence of an analysis request

4.2.4 Class and component design

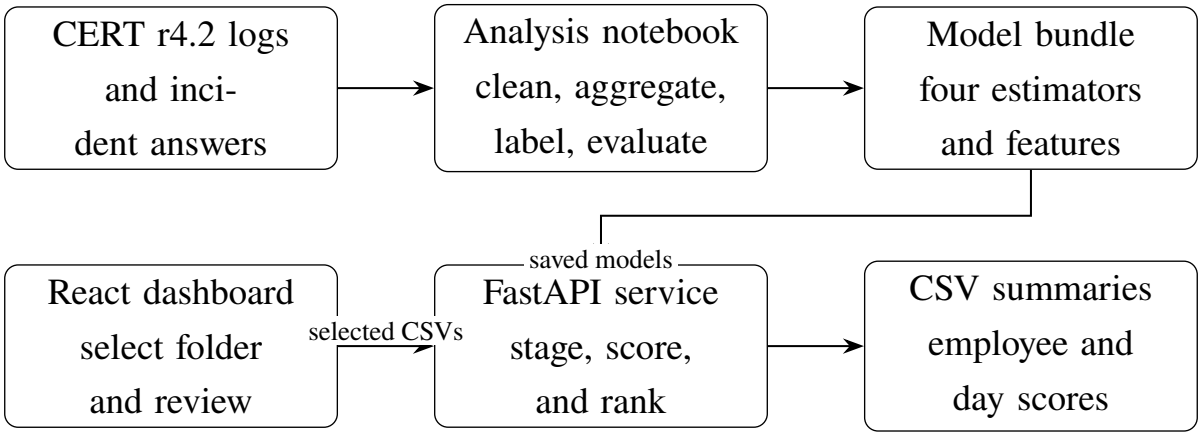


Figure 4.4: Component design and data flow of the project

4.3 Project pipeline and output design

The training notebook writes a model bundle containing the estimator dictionary, feature names, and model names. The FastAPI service loads this bundle, creates the same daily numeric feature columns from the five input sources, fills absent columns with zero, and calculates four model scores. Scores are converted to within-dataset percentile ranks and averaged for a consensus employee ranking. The dashboard marks the top five percent of users by consensus and users in the top ten percent for at least two models as "SUSPICIOUS"; remaining users are marked "REVIEW". These are application-specific review labels, not validated probability thresholds.

Generated CSV reports are saved under `sprint1_output`. There is no database in the current implementation. CSV and joblib files are local artifacts, so file permissions, encryption, backup, and deletion must be managed by the operator.

4.4 Database design

No relational database is implemented. The logical data entities are: ActivityEvent (source row), EmployeeDay (user, day, feature counts, target label for evaluation), ModelArtifact (model names and feature columns), EmployeeRanking (user, relative scores, review status), and ReviewSignal (reason and supporting high-ranked activity day). CSV is used for tabular interchange, and joblib serializes Python estimators. A production version should define explicit schemas, provenance, retention controls, and access logs before introducing persistent case data.

4.5 UI design

The primary interface is a React dashboard served locally by Vite and connected to a FastAPI service on the same computer. The landing view includes the activity-folder picker, model names, a human-review caution, and an expandable guide that shows how the five activity sources become employee-day features, pass through four independent models, and become a percentile-ranked queue. The guide explains the distinct roles of Isolation Forest, Logistic Regression, Decision Tree, and Random Forest. After analysis, the dashboard presents summary counts, prioritized review cards, a searchable and filterable employee directory, model-agreement signals, relative model ranks, and each selected employee's highest-ranked activity days. Explanations use aggregated counts and timing signals rather than message bodies, URLs, or file contents. The visual design uses a dark navy sidebar, cool neutral surfaces, and distinct restrained accents for the four model views. The selected CSV files are sent to the local API, staged in a temporary folder, and removed from that staging folder after analysis; the generated dashboard summaries are saved under `sprint1_output`. The previous Streamlit interface remains available as an alternative.

During analysis, the React interface displays an animated model motif, the upload and analysis stage, and a percentage progress bar. Upload progress comes from the browser's file-transfer events; the API reports milestones as each activity source and model is processed. The interface also displays an approximate remaining-time estimate extrapolated from elapsed time and reported progress. Because progress is reported at processing milestones and work per milestone can vary with file size, the remaining time is indicative rather than a guaranteed completion time. A note beside the review queue explains that model agreement counts how many model rankings place an employee among their respective top ten percent and that agreement is a review-prioritization signal, not a probability or proof of misconduct.

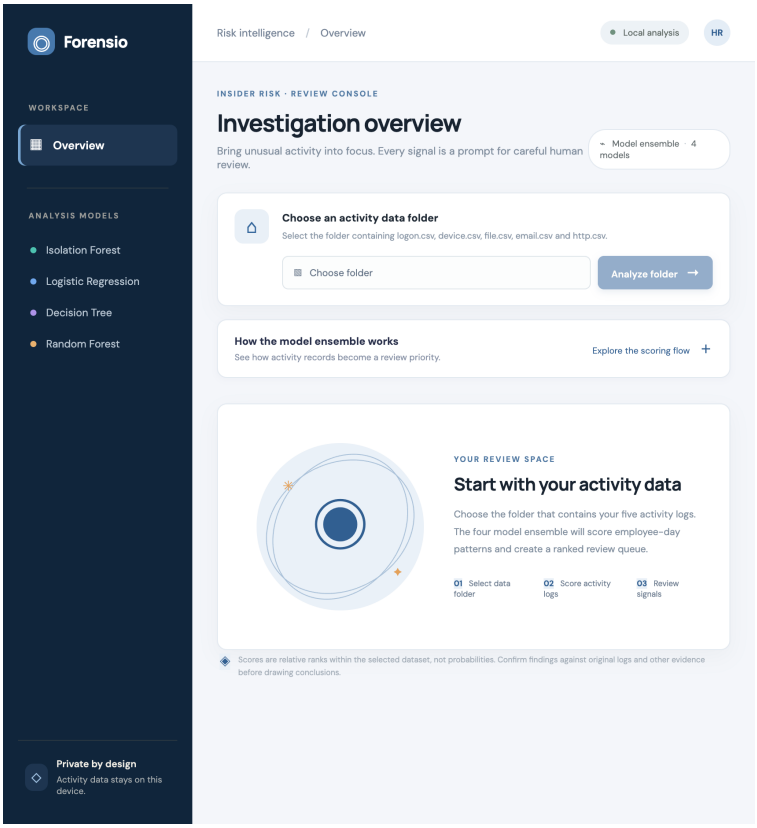


Figure 4.5: Landing screen with folder selection and the review guide collapsed.

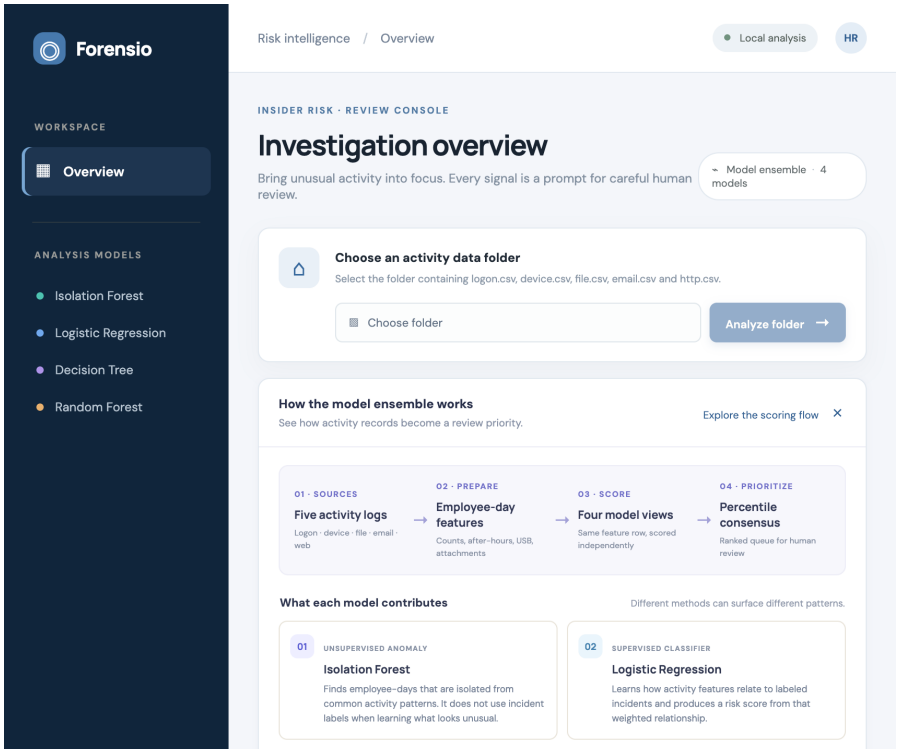


Figure 4.6: Expanded scoring guide with model descriptions and the review rule.

RESULTS AND DISCUSSION

5.1 Testing approach

The repository contains no dedicated automated test suite. Verification for this report consisted of reading the actual source and saved artifacts, checking the saved schema and quality summaries, running the grouped split and model evaluation against the saved labelled CSV, and checking Python syntax for notebook code cells. The dashboard was not subjected to a documented end-to-end analyst acceptance test in this report. The React landing screen and expanded model guide were captured while the local API reported the model bundle as ready; no folder analysis was run for those screenshots. Table 5.1 distinguishes completed checks from recommended manual tests.

Table 5.1: Test cases and verification status

ID	Test / expected behavior	Status
T01	Parse all notebook code cells; no syntax errors	Passed
T02	Verify saved labelled table dimensions, users, date span, and positive count	Passed
T03	Verify grouped train/validation/test partitions have positive labels and no user overlap	Passed
T04	Refit the four configured estimators and calculate validation/test metrics on saved data	Passed
T05	Confirm test predictions and confusion-matrix totals at the validation-selected capacity	Passed
T06	Select a folder missing a required CSV; application should display a clear error	Recommended manual test
T07	Analyze a valid folder and compare dashboard feature totals with notebook output	Recommended manual test
T08	Test malformed dates, blank users, duplicate rows across chunk boundaries, and empty files	Recommended automated tests
T09	Verify output permissions and deletion/retention for sensitive logs	Not implemented

5.2 Model planning and evaluation protocol

The saved evaluation artifact records ranking and fixed-capacity metrics. Average Precision selects the model. A review rate is selected by validation F1 for that model from the candidate capacities 1%, 2%, 3%, 5%, 7.5%, 10%, 15%, and 20%; test data is not used to choose the rate. Test metrics are reported after the model and operating capacity are fixed. The saved split groups records by user and contains 198,259 training rows with 815 positive labels, 66,075 validation rows with 260 positive labels, and 66,118 test rows with 289 positive labels. The three user sets are disjoint.

The Random Forest augmentation creates count-perturbed variants of positive training employee-days using source-consistent Poisson and binomial resampling. It creates one synthetic row per positive training row (815 rows for the selected split). Synthetic rows are used only in model fitting; validation and test data remain real and unchanged. For each partition, scores are sorted to select the chosen proportion. This is a ranking-based operating point, not a fixed score threshold. The selected one-percent test queue contains 662 employee-days; the five-percent comparison queue contains 3,306. Metrics are rounded in the tables, so counts from the confusion matrices are authoritative.

5.3 Comparison of models

At the validation-selected one-percent review capacity, the results are:

Table 5.2: Metrics at the 1% queue rate selected by validation F1

Model	Validation F1	Test P@1%	Test R@1%	Test F1	Test AP
Random Forest	0.254	0.144	0.329	0.200	0.158
Decision Tree	0.185	0.121	0.277	0.168	0.104
Logistic Regression	0.011	0.003	0.007	0.004	0.027
Isolation Forest	0.015	0.027	0.062	0.038	0.027

The augmented Random Forest remains selected because it has the highest validation AP (0.275). Its validation AP is higher than the real-data Random Forest baseline (0.214), so validation selects the augmented variant. On the held-out test split, the augmented model has AP 0.158 versus 0.088 for the baseline, and F1 0.200 versus 0.130 at the validation-selected one-percent capacity. At five percent, augmented Random Forest F1 is 0.099; moving to one percent raises precision from 5.38% to 14.35% while recall falls from 61.59% to 32.87%. Average Precision is unchanged by the queue cutoff. Decision Tree's higher test F1 is reported for comparison only and is not used to select the model or capacity.

The original five-percent results remain useful as a larger-capacity reference:

Table 5.3: Model evaluation from saved `model_comparison.csv`

Model	Validation AP	Validation P@5%	Validation R@5%	Test P@5%	Test R@5%
Random Forest	0.275	0.060	0.758	0.054	0.616
Decision Tree	0.185	0.055	0.704	0.035	0.405
Logistic Regression	0.027	0.037	0.465	0.037	0.419
Isolation Forest	0.016	0.008	0.096	0.018	0.208

Table 5.4: Additional test metrics from saved evaluation

Model	Test AP	Test F1 @ 5%	Test accuracy @ 5%
Random Forest	0.158	0.099	0.951
Decision Tree	0.104	0.065	0.949
Logistic Regression	0.027	0.067	0.949
Isolation Forest	0.027	0.033	0.947

The augmented Random Forest was selected by the highest validation Average Precision (0.275). The real-data Random Forest baseline had test AP 0.088 and the validation-selected

augmented model had test AP 0.158. Decision Tree's test AP is 0.104. Model selection used validation AP; test results are reported once as estimates and were not used to choose the augmentation. The queue rate is also selected on the validation partition only; test AP does not change when the review rate changes.

5.4 Evaluation metrics

Precision at 5% is the fraction of the fixed-size top-ranked queue matching known positive labels. **Recall at 5%** is the fraction of all known positive test employee-days found in that queue. **Average Precision (AP)** measures ranking performance across possible score thresholds. **F1** combines positive-class precision and recall at the selected queue rule. **Accuracy** is the fraction of all test employee-days correctly classified at that same rule. Since only 289 of 66,118 test records are labelled positive (0.437%), accuracy and weighted averages can look strong even when positive-class precision is low. Macro average gives equal weight to class-level scores but should not replace inspection of each class.

For a selected queue size k, let Q_k be the set of the k highest-scored test employee-days and let P be the set of all employee-days labelled as overlapping a known incident window. The reported capacity metrics are:

$$\text{Precision@ } k = \frac{|Q_k \cap P|}{k}, \qquad \text{Recall@ } k = \frac{|Q_k \cap P|}{|P|}.$$

For the capacity-reference results, $k = \lceil 0.05N \rceil$. The validation-selected operating point instead uses $k = \lceil 0.01N \rceil$. These definitions make the review-capacity assumption explicit: increasing k increases the number of records presented for review and can change both precision and recall. Both rates are recalculated within each partition and should not be mistaken for transferable probability thresholds.

For the dashboard, let $r_{u,d,m}$ be model m's percentile rank for user u on day d. The interface averages the four model percentile ranks for each employee-day and then uses that employee's maximum daily consensus value for its employee ranking:

$$c_{u,d} = \frac{1}{4} \sum_{m=1}^{4} r_{u,d,m}, \qquad C_u = \max_d c_{u,d}.$$

This is an application ranking rule, not a calibrated probability and not the model-selection rule used in the offline evaluation. The dashboard also uses a top-five-percent consensus cutoff and a two-of-four model-agreement condition for its "SUSPICIOUS" display status; those employee-level rules have not been evaluated by the employee-day test metrics in this report.

5.5 Confusion matrix

The integer counts in Table 5.5 and Appendix 10.3 show the validation-selected one-percent operating point. Five-percent performance is summarized in Table 5.3.

At the selected one-percent capacity, the augmented Random Forest places 95 known

Table 5.5: Random Forest confusion matrix at the validation-selected 1% queue rate

	Predicted not listed	Predicted in known window
Actual not listed	65,262	567
Actual in known window	194	95

positive employee-days in the 662-record queue and leaves 194 known positives outside it. The other 567 records do not match a known label; they are not proven benign. At the five-percent reference capacity, the model places 178 known positives in 3,306 records and leaves 111 known positives outside. The lower-capacity option improves test F1 in this synthetic experiment while retrieving fewer known positives overall.

5.6 Discussion

The model comparison illustrates that unsupervised anomaly detection does not necessarily rank known incident examples well: Isolation Forest has test recall 20.8% and precision 1.8% at the same queue size. The supervised Random Forest catches more known positives in the saved test partition after training-only synthetic augmentation. Its 14.4% precision still means most queued records do not match a known incident label. The Decision Tree has test AP 0.104, below augmented Random Forest's 0.158, and its validation AP is lower. This difference emphasizes why validation-based selection and a one-time test evaluation should be kept separate.

5.7 Requirements traceability

Table 5.6: Requirements traceability

ID	Requirement	Implementation / evidence
R01	Read five activity sources	React folder picker and `api.py`; notebook ingestion
R02	Create employee-day numeric features	notebook and `process_activity_file`
R03	Keep answer labels out of inference upload	answer file used in notebook only
R04	Compare four models	saved comparison table and bundle
R05	Hold user groups apart in evaluation	nested grouped train/validation/test split
R06	Present human-review context	React report, activity indicators, model guide, and caution text
R07	Establish real-world validity and operational threshold	Not satisfied; future work

MODEL DEPLOYMENT

6.1 Local demonstration setup

The primary React demonstration runs a local FastAPI service and Vite client. From the project root, start the API:

```
./.forensic/bin/python -m pip install -r requirements.txt
./.forensic/bin/python -m uvicorn api:app --host 127.0.0.1 --port 8000
```

In a second terminal, start the client:

```
cd frontend
npm install
npm run dev
```

Open the Vite URL (normally `http://127.0.0.1:5173`) and select a folder containing `logon.csv`, `device.csv`, `file.csv`, `email.csv`, and `http.csv`. The model bundle must be at `sprint1_output/insider_investigation_model.joblib`; run notebook Cell 31 if it is missing. The API stages uploads temporarily, processes files in 25,000-row chunks, removes the staged copies, and writes `dashboard_report.csv` and `dashboard_day_scores.csv` under `sprint1_output`.

The optional Streamlit interface starts from the project root with:

```
./.forensic/bin/streamlit run app.py
```

6.2 Analyst workflow and limitations

1. Confirm the model bundle is available, select the five-file activity folder, and run "Analyze folder". The interface shows upload and processing progress; missing required files stop the run.
2. Inspect employee rankings, model agreement, activity signals, and the highest-ranked days. Treat these as review leads and corroborate them against authorized source records and context.
3. Handle the generated CSVs under the same access and retention rules as the input logs.

This local proof of concept has no analyst authentication, role controls, audit trail, encrypted storage, retention enforcement, or case workflow; local file permissions remain the operator's responsibility. Load only trusted joblib bundles. The Streamlit alternative uses a macOS AppleScript folder picker.

GIT HISTORY

The following selected entries are taken from the repository history. They record the sequence of notebook and application changes visible in Git; the entries are not a complete project diary.

Table 7.1: Selected repository history

Commit	Date	Recorded message
da720e68	2026-10-11	Refactor code structure for improved readability and maintainability
8b99ad0e	2026-10-11	Implement code changes to enhance functionality and improve performance
c7b28395	2026-10-10	Fix execution counts and update table IDs in forensic notebook; resolve NameError in classification report output
513f90d8	2026-10-10	Fix NameError in classification report output and add empty code cell

Implementation challenges include processing a very large web-activity file, aligning activity from multiple sources to one employee-day record, handling severe class imbalance, selecting a model without using the test set for selection, and presenting scores without implying that a flag establishes wrongdoing. The primary working review demonstration uses a React client and a local FastAPI service; a Streamlit alternative remains in the repository.

CONCLUSIONS

8.1 Learning Outcomes

This project provided practical experience with a complete tabular machine-learning workflow: data inspection, chunked CSV processing, feature aggregation, label construction, grouped model evaluation, class-imbalance-aware metrics, model serialization, a React/Vite client, a FastAPI service, and local dashboard development. It also highlights the importance of distinguishing a known-positive label from an unverified negative, separating validation from test data, and communicating uncertainty in an investigation support tool.

8.2 Technical Skills Acquired

The implementation applies Python, pandas, NumPy, scikit-learn, Jupyter Notebook, joblib, FastAPI, React, Vite, Streamlit, Git, and CSV-based data interchange. It uses Logistic Regression, Decision Tree, Random Forest, and Isolation Forest; grouped train/validation/test splits; Average Precision; precision, recall, and F1; and confusion-matrix analysis.

8.3 Analytical and Professional Learning

The project demonstrates that a high overall accuracy is not sufficient for a rare-event use case. Review capacity, false-alert burden, incomplete labels, and the unit of evaluation must be considered together. The dashboard must be used as a human-review aid, with original records and independent context checked before drawing conclusions. These are project-derived learning points; this report does not make claims about unverified internship activities or personal performance.

8.4 Summary of the Project

This project demonstrates a local pipeline from activity logs to employee-day features, model evaluation, and a React/FastAPI review dashboard. The validation-selected Random Forest's one-percent test queue retrieves 95 of 289 known positives. These results rank synthetic records for review; incomplete labels and low precision mean they do not prove harmful behaviour or validate the dashboard's employee-level status. Any operational use requires authorization, corroborating evidence, and accountable human review.

FUTURE SCOPE

1. Define the analyst's real review capacity and acceptable minimum precision with project stakeholders; compare several queue sizes on validation data.
2. Add repeated group-based cross-validation and uncertainty intervals so one split does not determine the conclusions.
3. Evaluate employee-level and time-forward generalization, ensuring that future activity cannot leak into earlier decisions.
4. Review incident-window construction with domain experts and distinguish confirmed negative examples from unlabeled records.
5. Add features for per-user baselines, work schedules, event sequences, and contextual deviations, subject to privacy and authorization review.
6. Perform false-alert case review and error analysis, while treating unmatched labels as unknown rather than automatically normal.
7. Add automated tests for malformed input, chunk-boundary duplicates, missing source columns, output schemas, and model feature alignment.
8. Add authenticated access, audit trails, secure storage, retention/deletion controls, model versioning, and documented analyst procedures before wider deployment.

APPENDIX

10.1 Minimum Software and Hardware Requirements

10.1.1 Minimum software requirements

- A supported Python installation with pandas, NumPy, scikit-learn, joblib, FastAPI, and Uvicorn.
- Node.js and npm to run the React/Vite client. Streamlit is needed only for the alternative interface.
- Jupyter Notebook or Visual Studio Code with Jupyter support to run the analysis notebook.
- A modern browser that supports selecting a folder and the local development setup. macOS is required only for the Streamlit alternative's AppleScript folder picker.
- The five input CSVs and the trained model bundle for the dashboard; the answer file is required to reproduce supervised training/evaluation, not for scoring a new activity folder.

10.1.2 Minimum hardware requirements

No measured minimum CPU, RAM, or free disk specification is available in the project. The operator needs enough memory and storage for the selected source logs, intermediate daily aggregates, and generated outputs. Because the HTTP file contains over 28 million rows in the provided run, available disk and memory should be checked before processing. Establish formal minimum hardware requirements through runtime and peak-memory measurements on the intended machine.

10.2 Output Artifact Guide

`model_comparison.csv` Validation and test metrics for all four models.

`confusion_matrix_counts.csv` Integer confusion-matrix values at the validation-selected queue rate.

`evaluation_employee_day_scores.csv` Test employee-day labels, raw scores, validation-selected one-percent queue flags, and top-five-percent reference flags.

`evaluation_employee_ranking.csv` Employee-level aggregation for the evaluation records.

`insider_investigation_model.joblib` Saved four-model bundle and ordered feature columns for the dashboard.

`dashboard_report.csv` Employee ranking generated by the interactive demonstration.

`dashboard_day_scores.csv` Employee-day scores generated by the interactive demonstration.

10.3 Selected Test Counts

FP denotes a predicted positive without a matching known-window label; given the incompleteness of the answer labels, this must not be interpreted as proof the record is benign.

Table 10.1: Confusion-matrix counts at the validation-selected 1% queue rate

Model	TN	FP (unmatched label)	FN	TP
Logistic Regression	65,169	660	287	2
Decision Tree	65,247	582	209	80
Random Forest	65,262	567	194	95
Isolation Forest	65,185	644	271	18

10.4 Responsible Use Checklist

1. Confirm lawful authorization and purpose before processing any activity data.
2. Restrict access to raw input and generated output files.
3. Present model outputs as leads, never as findings of misconduct.
4. Review original source records and independent context before taking action.
5. Document analyst decisions and allow correction of inaccurate or incomplete records.
6. Define a review capacity, retention period, and deletion process.
7. Assess bias, disparate impact, and performance drift before any operational deployment.

10.5 Selected Code Snippets

These excerpts illustrate daily feature aggregation, incident-window labelling, employee-grouped evaluation, API file staging, model scoring, and consensus ranking. They are selected portions of the current implementation; the full source files remain in the project repository. The separate Streamlit app listing is an alternative interface.

Listing 10.1: Daily event features aggregated by user and date

```
chunk["after_hours_count"] = (
    (hour < 6) | (hour >= 18)
).astype("int8").to_numpy()

chunk["day"] = dates.dt.strftime("%Y-%m-%d").to_numpy()
chunk["event_count"] = 1

feature_columns = ["event_count", "after_hours_count"]

if source == "device" and "activity" in chunk.columns:
    chunk["device_connect_count"] = (
        chunk["activity"].astype("string").str.lower() == "connect"
    ).astype("int8")
    feature_columns.append("device_connect_count")

if source == "email" and "attachments" in chunk.columns:
    chunk["attachment_count"] = (
        pd.to_numeric(chunk["attachments"], errors="coerce")
        .fillna(0)
    )
    feature_columns.append("attachment_count")

group = chunk.groupby(["user", "day"])[feature_columns].sum()
group = group.rename(
    columns={column: f"{source}_{column}" for column in feature_columns}
)
daily_parts.append(group)
```

Listing 10.2: Known incident-window labels matched to employee-days

```
employee_daily["day"] = pd.to_datetime(employee_daily["day"]).dt.normalize()

employee_daily["known_incident_window"] = 0
employee_daily["scenario"] = "not_listed"

for incident in incidents.itertuples(index=False):
    start_day = incident.start.normalize()
    end_day = incident.end.normalize()

    in_incident_window = (
        employee_daily["user"].eq(incident.user)
        & employee_daily["day"].between(start_day, end_day)
    )

    employee_daily.loc[in_incident_window, "known_incident_window"] = 1
    employee_daily.loc[in_incident_window, "scenario"] = str(incident.scenario)
```

Listing 10.3: Local API staging and cleanup for selected activity files

```
        "message": "Model␣bundle␣is␣ready." if MODEL_PATH.is_file() else "Model␣bundle␣is␣missing.␣Run␣
     notebook␣Cell␣31.",
    }


@app.get("/api/progress")
def progress():
    return analysis_progress


@app.post("/api/analyze")
def analyze(request: AnalyzeRequest):
    folder = Path(request.folder).expanduser()
    if not folder.is_dir():
        raise HTTPException(status_code=400, detail="That␣folder␣could␣not␣be␣found␣on␣this␣computer.")
    return _analyze_folder(folder)


@app.post("/api/analyze-upload")
def analyze_upload(files: list[UploadFile] = File(...)):
    expected = set(REQUIRED_FILES)
    selected: dict[str, UploadFile] = {}
    optional = {"employees.csv", "employee.csv", "users.csv", "user.csv"}
    for upload in files:
        name = Path(upload.filename or "").name.lower()
        if name in expected or name in optional:
```

Listing 10.4: Four-model scoring, percentile alignment, and consensus ranking

```
def _score(model, model_name, values):
    """Return␣larger␣values␣for␣higher␣risk␣or␣more␣unusual␣activity."""
    if model_name == "Isolation␣Forest":
        return -np.asarray(model.decision_function(values), dtype=float)
    if hasattr(model, "predict_proba"):
        return np.asarray(model.predict_proba(values)[:, 1], dtype=float)
    return np.asarray(model.decision_function(values), dtype=float)


def score_folder(data_folder, bundle, progress_callback=None):
    """Build␣employee-day␣features,␣score␣with␣all␣four␣models,␣and␣rank␣employees."""
    def report_progress(percent, message):
```

```
        if progress_callback is not None:
            progress_callback(percent, message)

    data_folder = Path(data_folder)
    missing = [name for name in REQUIRED_FILES if not (data_folder / name).is_file()]
    if missing:
        raise FileNotFoundError("Selected folder is missing: " + ", ".join(missing))

    sources = []
    for index, name in enumerate(REQUIRED_FILES):
        report_progress(index * 12, f"Reading and aggregating {name} ({index + 1} of {len(REQUIRED_FILES)})
     ")
        sources.append(process_activity_file(data_folder / name))
        report_progress((index + 1) * 12, f"Finished {name} ({index + 1} of {len(REQUIRED_FILES)})")
    sources = [frame for frame in sources if not frame.empty]
    if not sources:
        raise ValueError("The selected activity files contained no usable records.")

    employee_days = pd.concat(sources, axis=1).fillna(0)
    employee_days = employee_days.groupby(level=["user", "day"]).sum().reset_index()
    report_progress(65, "Preparing employee-day features")

    feature_columns = bundle["feature_columns"]
    for column in feature_columns:
        if column not in employee_days.columns:
            employee_days[column] = 0
    model_input = employee_days[feature_columns].apply(pd.to_numeric, errors="coerce").fillna(0)

    score_columns = []
```

10.6 Complete Source Listings

The following listings provide a fuller record of the implementation than the selected excerpts above. They are generated text snapshots of the project files used during report preparation. The notebook listing contains the analysis workflow, while the API, core, and Streamlit listings document the local interfaces and scoring implementation. Including these listings makes the appendix useful for code review and reproduction without treating source volume as experimental evidence.

10.6.1 Analysis notebook source

The notebook source covers input inspection, chunked data processing, feature aggregation, incident-window labeling, grouped splitting, estimator fitting, evaluation, and artifact generation. Notebook cell boundaries are represented in the exported listing where present.

Listing 10.5: Complete exported analysis notebook source

```
# Notebook code cell 2
from pathlib import Path
import json

import pandas as pd
import matplotlib.pyplot as plt

# Notebook code cell 4
project_dir = Path.cwd()

if project_dir.name == "notebook":
```

```
    project_dir = project_dir.parent

data_dir = project_dir / "r4.2"
output_dir = project_dir / "sprint1_output"
output_dir.mkdir(exist_ok=True)

required_files = [
    "logon.csv",
    "device.csv",
    "file.csv",
    "email.csv",
    "http.csv",
]

print("Project folder:", project_dir)
print("Data folder:", data_dir)
print("Output folder:", output_dir)

# Notebook code cell 5
missing_files = [
    name for name in required_files
    if not (data_dir / name).is_file()
]

if missing_files:
    print("These files are missing:", missing_files)
else:
    print("All five activity files are present.")

# Notebook code cell 7
for filename in required_files:
    file_path = data_dir / filename
    sample = pd.read_csv(file_path, nrows=5)

    print(f"\n{filename}")
    print("Columns:", list(sample.columns))
    display(sample.head(3))

# Notebook code cell 9
chunk_size = 25_000

def process_activity_file(filename):
    file_path = data_dir / filename
    source = file_path.stem.lower()

    # Read the header first so we can select only useful columns.
    available_columns = pd.read_csv(file_path, nrows=0).columns.tolist()
    available_columns = [column.strip().lower() for column in available_columns]

    useful_columns = ["date", "user", "pc", "activity", "attachments", "filename"]
    columns_to_read = [
        column for column in useful_columns
        if column in available_columns
    ]

    if "date" not in columns_to_read or "user" not in columns_to_read:
        raise ValueError(f"{filename} needs both 'date' and 'user' columns.")

    daily_parts = []
    quality = {
        "rows_read": 0,
```

```
        "rows_kept": 0,
        "missing_user": 0,
        "invalid_date": 0,
        "duplicates_removed": 0,
    }

    for chunk in pd.read_csv(
        file_path,
        usecols=lambda column: column.strip().lower() in columns_to_read,
        chunksize=chunk_size,
        low_memory=False,
    ):
        chunk.columns = [column.strip().lower() for column in chunk.columns]
        quality["rows_read"] += len(chunk)

        chunk["user"] = chunk["user"].astype("string").str.strip()
        missing_user = chunk["user"].isna() | chunk["user"].eq("")
        quality["missing_user"] += int(missing_user.sum())

        dates = pd.to_datetime(chunk["date"], errors="coerce", format="mixed")
        invalid_date = dates.isna()
        quality["invalid_date"] += int(invalid_date.sum())

        duplicate = chunk.duplicated()
        quality["duplicates_removed"] += int(duplicate.sum())

        keep = ~(missing_user | invalid_date | duplicate)
        chunk = chunk.loc[keep].copy()
        dates = dates.loc[keep]

        if chunk.empty:
            continue

        quality["rows_kept"] += len(chunk)

        # A row is outside normal working hours if it is before 6 AM
        # or at/after 6 PM, based on the timestamp in the dataset.
        hour = dates.dt.hour
        chunk["after_hours_count"] = (
            (hour < 6) | (hour >= 18)
        ).astype("int8").to_numpy()

        chunk["day"] = dates.dt.strftime("%Y-%m-%d").to_numpy()
        chunk["event_count"] = 1

        feature_columns = ["event_count", "after_hours_count"]

        if source == "device" and "activity" in chunk.columns:
            chunk["device_connect_count"] = (
                chunk["activity"].astype("string").str.lower() == "connect"
            ).astype("int8")
            feature_columns.append("device_connect_count")

        if source == "email" and "attachments" in chunk.columns:
            chunk["attachment_count"] = (
                pd.to_numeric(chunk["attachments"], errors="coerce")
                .fillna(0)
            )
            feature_columns.append("attachment_count")

        group = chunk.groupby(["user", "day"])[feature_columns].sum()
```

```
        group = group.rename(
            columns={column: f"{source}_{column}" for column in feature_columns}
        )
        daily_parts.append(group)

    if daily_parts:
        daily_features = (
            pd.concat(daily_parts)
            .groupby(level=["user", "day"])
            .sum()
        )
    else:
        daily_features = pd.DataFrame()

    return daily_features, quality

# Notebook code cell 10
daily_features_by_source = []
data_quality = {}

for filename in required_files:
    print(f"Processing {filename}...")
    source_features, source_quality = process_activity_file(filename)

    daily_features_by_source.append(source_features)
    data_quality[filename] = source_quality

    print(
        f"  Kept {source_quality['rows_kept']:,} "
        f"of {source_quality['rows_read']:,} rows"
    )

# Notebook code cell 12
employee_daily = pd.concat(daily_features_by_source, axis=1).fillna(0)
employee_daily = employee_daily.groupby(level=["user", "day"]).sum()
employee_daily = employee_daily.reset_index()

employee_daily.head()

# Notebook code cell 13
features_path = output_dir / "employee_daily_features.csv"
quality_path = output_dir / "data_quality_summary.json"

employee_daily.to_csv(features_path, index=False)
quality_path.write_text(
    json.dumps(data_quality, indent=2),
    encoding="utf-8",
)

print("Saved cleaned features to:", features_path)
print("Saved data-quality summary to:", quality_path)
print("Employee-day rows:", f"{len(employee_daily):,}")
print("Employees:", f"{employee_daily['user'].nunique():,}")

# Notebook code cell 15
event_columns = [
    column for column in employee_daily.columns
    if column.endswith("_event_count")
]

events_by_source = employee_daily[event_columns].sum().sort_values(
```

```
    ascending=False
)

display(events_by_source.to_frame("event_count"))

events_by_source.plot(
    kind="bar",
    figsize=(9, 5),
    title="Recorded activity by source",
    ylabel="Event count",
    xlabel="Activity source",
    color="steelblue",
)

plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()

# Notebook code cell 16
employee_daily["day"] = pd.to_datetime(employee_daily["day"])

daily_event_totals = (
    employee_daily.groupby("day")[event_columns]
    .sum()
    .sum(axis=1)
)

daily_event_totals.plot(
    figsize=(12, 5),
    title="Daily activity volume",
    ylabel="Event count",
    xlabel="Date",
    color="darkorange",
)

plt.tight_layout()
plt.show()

# Notebook code cell 17
employee_event_totals = (
    employee_daily.groupby("user")[event_columns]
    .sum()
    .sum(axis=1)
    .sort_values(ascending=False)
)

top_employees = employee_event_totals.head(10).rename("event_count")
display(top_employees.to_frame())

top_employees.plot(
    kind="bar",
    figsize=(10, 5),
    title="Employees with the most recorded events",
    ylabel="Event count",
    xlabel="Employee ID",
    color="slateblue",
)

plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

```
# Notebook code cell 20
answers_file = project_dir / "answers" / "insiders.csv"

incidents = pd.read_csv(answers_file)

incidents = incidents[incidents["dataset"].astype(str) == "4.2"].copy()

incidents["start"] = pd.to_datetime(
    incidents["start"],
    format="%m/%d/%Y␣%H:%M:%S",
    errors="coerce",
)

incidents["end"] = pd.to_datetime(
    incidents["end"],
    format="%m/%d/%Y␣%H:%M:%S",
    errors="coerce",
)

incidents = incidents.dropna(subset=["start", "end"])
incidents.head()

# Notebook code cell 21
employee_daily["day"] = pd.to_datetime(employee_daily["day"]).dt.normalize()

employee_daily["known_incident_window"] = 0
employee_daily["scenario"] = "not_listed"

for incident in incidents.itertuples(index=False):
    start_day = incident.start.normalize()
    end_day = incident.end.normalize()

    in_incident_window = (
        employee_daily["user"].eq(incident.user)
        & employee_daily["day"].between(start_day, end_day)
    )

    employee_daily.loc[in_incident_window, "known_incident_window"] = 1
    employee_daily.loc[in_incident_window, "scenario"] = str(incident.scenario)

print("Known␣incident␣cases:", len(incidents))
print("Employee-days:", len(employee_daily))
print(
    "Employee-days␣inside␣known␣incident␣windows:",
    int(employee_daily["known_incident_window"].sum()),
)

# Notebook code cell 22
label_counts = employee_daily["known_incident_window"].value_counts().sort_index()
label_counts.index = ["Not␣listed␣in␣incident␣windows", "Inside␣incident␣window"]
display(label_counts.to_frame("employee_day_count"))

labeled_file = output_dir / "employee_daily_labeled.csv"
employee_daily.to_csv(labeled_file, index=False)

print("Saved␣labeled␣employee-days␣to:", labeled_file)

# Notebook code cell 24
import numpy as np
```

```
from sklearn.ensemble import IsolationForest, RandomForestClassifier
from sklearn.base import clone
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score
from sklearn.model_selection import StratifiedGroupKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier

# Notebook code cell 25
target_column = "known_incident_window"

# Do not let employee IDs, dates, scenarios, or the answer label become model inputs.
excluded_columns = {
    "user",
    "day",
    "scenario",
    target_column,
}

feature_columns = [
    column for column in employee_daily.select_dtypes(include="number").columns
    if column not in excluded_columns
]

X = employee_daily[feature_columns].fillna(0)
y = employee_daily[target_column].astype(int)
groups = employee_daily["user"]

print("Features:", feature_columns)
print("Employee-days:", len(employee_daily))
print("Incident-window labels:", int(y.sum()))

# Hold out employees, not individual days, to avoid the same employee appearing
# in both model development and evaluation data.
outer_splitter = StratifiedGroupKFold(
    n_splits=5,
    shuffle=True,
    random_state=42,
)

train_validation_indices, test_indices = next(
    outer_splitter.split(X, y, groups=groups)
)

inner_splitter = StratifiedGroupKFold(
    n_splits=4,
    shuffle=True,
    random_state=43,
)
train_relative_indices, validation_relative_indices = next(
    inner_splitter.split(
        X.iloc[train_validation_indices],
        y.iloc[train_validation_indices],
        groups=groups.iloc[train_validation_indices],
    )
)
train_indices = train_validation_indices[train_relative_indices]
validation_indices = train_validation_indices[validation_relative_indices]
```

```
X_train = X.iloc[train_indices]
X_validation = X.iloc[validation_indices]
X_test = X.iloc[test_indices]
y_train = y.iloc[train_indices]
y_validation = y.iloc[validation_indices]
y_test = y.iloc[test_indices]

train_users = set(groups.iloc[train_indices])
validation_users = set(groups.iloc[validation_indices])
test_users = set(groups.iloc[test_indices])

print("Training employees:", len(train_users))
print("Validation employees:", len(validation_users))
print("Test employees:", len(test_users))
print("Training positive employee-days:", int(y_train.sum()))
print("Validation positive employee-days:", int(y_validation.sum()))
print("Test positive employee-days:", int(y_test.sum()))
print(
    "Employees shared between splits:",
    len(train_users & validation_users)
    + len(train_users & test_users)
    + len(validation_users & test_users),
)

if min(y_train.sum(), y_validation.sum(), y_test.sum()) == 0:
    raise ValueError("Each grouped split must contain incident-window examples.")

# Notebook code cell 26
models = {
    "Logistic Regression": make_pipeline(
        SimpleImputer(strategy="median"),
        StandardScaler(),
        LogisticRegression(
            max_iter=1000,
            class_weight="balanced",
            random_state=42,
        ),
    ),
    "Decision Tree": DecisionTreeClassifier(
        max_depth=10,
        min_samples_leaf=10,
        class_weight="balanced",
        random_state=42,
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        min_samples_leaf=5,
        class_weight="balanced_subsample",
        n_jobs=-1,
        random_state=42,
    ),
    # This model learns what is unusual without using the incident labels.
    "Isolation Forest": IsolationForest(
        n_estimators=200,
        contamination="auto",
        random_state=42,
        n_jobs=-1,
    ),
}
```

```
def make_synthetic_positive_counts(X_source, y_source, random_state=2026):
    """Create one count-perturbed positive example per real positive training row."""
    rng = np.random.default_rng(random_state)
    values = X_source.to_numpy(dtype=float)
    positive_rows = values[np.asarray(y_source) == 1]
    if len(positive_rows) == 0:
        raise ValueError("Synthetic augmentation needs positive training examples.")

    anchors = positive_rows[rng.integers(0, len(positive_rows), size=len(positive_rows))]
    synthetic = np.zeros_like(anchors)
    scale = rng.lognormal(mean=0.0, sigma=0.12, size=len(anchors))
    columns = list(X_source.columns)

    # Resample event counts first, then preserve the source-level count relationships.
    for source in ("logon", "device", "file", "email", "http"):
        event_column = f"{source}_event_count"
        if event_column in columns:
            event_index = columns.index(event_column)
            synthetic[:, event_index] = rng.poisson(
                np.maximum(anchors[:, event_index], 0) * scale
            )

        after_hours_column = f"{source}_after_hours_count"
        if event_column in columns and after_hours_column in columns:
            after_index = columns.index(after_hours_column)
            event_index = columns.index(event_column)
            rate = np.divide(
                anchors[:, after_index],
                anchors[:, event_index],
                out=np.zeros(len(anchors), dtype=float),
                where=anchors[:, event_index] > 0,
            ).clip(0, 1)
            synthetic[:, after_index] = rng.binomial(
                synthetic[:, event_index].astype(int), rate
            )

    connect_column = "device_device_connect_count"
    device_events = "device_event_count"
    if connect_column in columns and device_events in columns:
        connect_index = columns.index(connect_column)
        device_index = columns.index(device_events)
        connect_rate = np.divide(
            anchors[:, connect_index],
            anchors[:, device_index],
            out=np.zeros(len(anchors), dtype=float),
            where=anchors[:, device_index] > 0,
        ).clip(0, 1)
        synthetic[:, connect_index] = rng.binomial(
            synthetic[:, device_index].astype(int), connect_rate
        )

    attachment_column = "email_attachment_count"
    if attachment_column in columns:
        attachment_index = columns.index(attachment_column)
        synthetic[:, attachment_index] = rng.poisson(
            np.maximum(anchors[:, attachment_index], 0) * scale
        )

    return pd.DataFrame(synthetic, columns=columns, index=None)
```

```
validation_risk_scores = {}
risk_scores = {}
selected_synthetic_augmentation = {name: False for name in models}
baseline_rf_model = None
baseline_rf_test_scores = None

for name, model in models.items():
    print(f"Training␣{name}...")
    if name == "Random␣Forest":
        # Keep a real-data baseline, then use validation AP to choose whether
        # one synthetic count-perturbed row per positive training row helps.
        baseline_rf_model = clone(model).fit(X_train, y_train)
        baseline_validation = baseline_rf_model.predict_proba(X_validation)[:, 1]
        synthetic_X = make_synthetic_positive_counts(X_train, y_train)
        augmented_X = pd.concat([X_train.reset_index(drop=True), synthetic_X], ignore_index=True)
        augmented_y = pd.concat(
            [y_train.reset_index(drop=True), pd.Series(np.ones(len(synthetic_X), dtype=int))],
            ignore_index=True,
        )
        augmented_model = clone(model).fit(augmented_X, augmented_y)
        augmented_validation = augmented_model.predict_proba(X_validation)[:, 1]
        if average_precision_score(y_validation, augmented_validation) > average_precision_score(
            y_validation, baseline_validation
        ):
            model = augmented_model
            selected_synthetic_augmentation[name] = True
            print(f"␣␣Validation␣selected␣synthetic␣augmentation␣({len(synthetic_X):,}␣generated␣rows).")
        else:
            model = baseline_rf_model
            print("␣␣Validation␣retained␣the␣real-data␣baseline.")
        baseline_rf_test_scores = baseline_rf_model.predict_proba(X_test)[:, 1]
    elif name == "Isolation␣Forest":
        model.fit(X_train)
    else:
        model.fit(X_train, y_train)

    models[name] = model
    if name == "Isolation␣Forest":
        validation_risk_scores[name] = -model.decision_function(X_validation)
        risk_scores[name] = -model.decision_function(X_test)
    else:
        validation_risk_scores[name] = model.predict_proba(X_validation)[:, 1]
        risk_scores[name] = model.predict_proba(X_test)[:, 1]

print("Model␣variants␣were␣selected␣using␣validation␣data␣only.")


# Notebook code cell 28
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    ConfusionMatrixDisplay,
)

# Choose a practical review capacity on validation data only. Selecting from a
# coarse grid is less sensitive to one exact cutoff than searching every row.
candidate_review_rates = [0.01, 0.02, 0.03, 0.05, 0.075, 0.10, 0.15, 0.20]
```

```
validation_labels = y_validation.to_numpy()
actual_labels = y_test.to_numpy()

# Preserve model selection by validation Average Precision.
selected_model_for_rate = max(
    validation_risk_scores,
    key=lambda name: average_precision_score(
        validation_labels, np.asarray(validation_risk_scores[name])
    ),
)
selected_model_validation_scores = np.asarray(
    validation_risk_scores[selected_model_for_rate]
)

best_rate = None
best_validation_f1 = -1.0
for candidate_rate in candidate_review_rates:
    candidate_count = max(
        1, int(np.ceil(len(validation_labels) * candidate_rate))
    )
    candidate_indices = np.argsort(selected_model_validation_scores)[-candidate_count:]
    candidate_predictions = np.zeros(len(validation_labels), dtype=int)
    candidate_predictions[candidate_indices] = 1
    candidate_f1 = f1_score(
        validation_labels, candidate_predictions, zero_division=0
    )
    # Rates are ascending, so retaining the first tie favors the smaller queue.
    if candidate_f1 > best_validation_f1:
        best_validation_f1 = candidate_f1
        best_rate = candidate_rate

selected_queue_rate = float(best_rate)
validation_top_count = max(
    1, int(np.ceil(len(validation_labels) * selected_queue_rate))
)
test_top_count = max(1, int(np.ceil(len(actual_labels) * selected_queue_rate)))
validation_five_percent_count = max(
    1, int(np.ceil(len(validation_labels) * 0.05))
)
test_five_percent_count = max(1, int(np.ceil(len(actual_labels) * 0.05)))

print(
    f"Selected␣model:␣{selected_model_for_rate};␣"
    f"validation-F1␣queue␣capacity:␣{selected_queue_rate:.1%}␣"
    f"({validation_top_count:,}␣of␣{len(validation_labels):,}␣employee-days);␣"
    f"validation␣F1:␣{best_validation_f1:.3f}"
)

results = []
confusion_matrices = {}
for name, test_scores in risk_scores.items():
    validation_scores = np.asarray(validation_risk_scores[name])
    test_scores = np.asarray(test_scores)

    validation_top_indices = np.argsort(validation_scores)[-validation_top_count:]
    validation_predictions = np.zeros(len(validation_labels), dtype=int)
    validation_predictions[validation_top_indices] = 1
    test_top_indices = np.argsort(test_scores)[-test_top_count:]
    test_predictions = np.zeros(len(actual_labels), dtype=int)
    test_predictions[test_top_indices] = 1
```

```
    # Keep the original 5% capacity as a reference for the precision/recall tradeoff.
    validation_five_indices = np.argsort(validation_scores)[-validation_five_percent_count:]
    validation_five_predictions = np.zeros(len(validation_labels), dtype=int)
    validation_five_predictions[validation_five_indices] = 1
    test_five_indices = np.argsort(test_scores)[-test_five_percent_count:]
    test_five_predictions = np.zeros(len(actual_labels), dtype=int)
    test_five_predictions[test_five_indices] = 1

    test_matrix = confusion_matrix(
        actual_labels, test_predictions, labels=[0, 1]
    )
    confusion_matrices[name] = test_matrix

    baseline_rf_top_indices = np.argsort(baseline_rf_test_scores)[-test_top_count:]
    baseline_rf_predictions = np.zeros(len(actual_labels), dtype=int)
    baseline_rf_predictions[baseline_rf_top_indices] = 1

    results.append({
        "Model": name,
        "Selected queue rate": selected_queue_rate,
        "Validation AP": average_precision_score(
            validation_labels, validation_scores
        ),
        "Validation precision @ selected queue": precision_score(
            validation_labels, validation_predictions, zero_division=0
        ),
        "Validation recall @ selected queue": recall_score(
            validation_labels, validation_predictions, zero_division=0
        ),
        "Validation F1 @ selected queue": f1_score(
            validation_labels, validation_predictions, zero_division=0
        ),
        "Test precision @ selected queue": precision_score(
            actual_labels, test_predictions, zero_division=0
        ),
        "Test recall @ selected queue": recall_score(
            actual_labels, test_predictions, zero_division=0
        ),
        "Test F1 @ selected queue": f1_score(
            actual_labels, test_predictions, zero_division=0
        ),
        "Test false alerts @ selected queue": int(test_matrix[0, 1]),
        "Test AP": average_precision_score(actual_labels, test_scores),
        "Validation precision @ 5%": precision_score(
            validation_labels, validation_five_predictions, zero_division=0
        ),
        "Validation recall @ 5%": recall_score(
            validation_labels, validation_five_predictions, zero_division=0
        ),
        "Test precision @ 5%": precision_score(
            actual_labels, test_five_predictions, zero_division=0
        ),
        "Test recall @ 5%": recall_score(
            actual_labels, test_five_predictions, zero_division=0
        ),
        "Test accuracy @ 5%": accuracy_score(actual_labels, test_five_predictions),
        "Test F1 @ 5%": f1_score(
            actual_labels, test_five_predictions, zero_division=0
        ),
        "Synthetic rows added": (
            int(y_train.sum()) if name == "Random Forest" and selected_synthetic_augmentation[name] else 0
```

```
        ),
        "RF baseline Test AP": (
            average_precision_score(actual_labels, baseline_rf_test_scores)
            if name == "Random Forest" else np.nan
        ),
        "RF baseline Test precision @ selected queue": (
            precision_score(actual_labels, baseline_rf_predictions, zero_division=0)
            if name == "Random Forest" else np.nan
        ),
        "RF baseline Test recall @ selected queue": (
            recall_score(actual_labels, baseline_rf_predictions, zero_division=0)
            if name == "Random Forest" else np.nan
        ),
        "RF baseline Test F1 @ selected queue": (
            f1_score(actual_labels, baseline_rf_predictions, zero_division=0)
            if name == "Random Forest" else np.nan
        ),
    })

# Select the model using validation AP; never use test metrics for selection.
model_comparison = (
    pd.DataFrame(results)
    .sort_values("Validation AP", ascending=False)
    .reset_index(drop=True)
)
display(model_comparison.style.format({
    "Selected queue rate": "{:.1%}",
    "Validation AP": "{:.3f}",
    "Validation precision @ selected queue": "{:.3f}",
    "Validation recall @ selected queue": "{:.3f}",
    "Validation F1 @ selected queue": "{:.3f}",
    "Test precision @ selected queue": "{:.3f}",
    "Test recall @ selected queue": "{:.3f}",
    "Test F1 @ selected queue": "{:.3f}",
    "Test AP": "{:.3f}",
    "Validation precision @ 5%": "{:.3f}",
    "Validation recall @ 5%": "{:.3f}",
    "Test precision @ 5%": "{:.3f}",
    "Test recall @ 5%": "{:.3f}",
    "Test accuracy @ 5%": "{:.3f}",
    "Test F1 @ 5%": "{:.3f}",
}))

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()
for ax, (name, matrix) in zip(axes, confusion_matrices.items()):
    display_matrix = ConfusionMatrixDisplay(
        confusion_matrix=matrix,
        display_labels=["Not in incident window", "In incident window"],
    )
    display_matrix.plot(ax=ax, colorbar=False, values_format="d")
    ax.set_title(f"{name}  {selected_queue_rate:.1%} queue")
    ax.tick_params(axis="x", labelrotation=25)
plt.tight_layout()
plt.show()


# Notebook code cell 30
# Save the model comparison, including the validation-selected queue rate.
comparison_path = output_dir / "model_comparison.csv"
model_comparison.to_csv(comparison_path, index=False)
```

```
# Preserve test labels and scores, plus flags at both selected and 5% capacity.
comparison_best_model = model_comparison.iloc[0]["Model"]
selected_queue_rate = float(model_comparison.iloc[0]["Selected queue rate"])
best_risk_column = comparison_best_model.lower().replace(" ", "_") + "_risk"
scored_days = employee_daily.iloc[test_indices][
    ["user", "day", target_column]
].copy().reset_index(drop=True)
actual_labels = y_test.to_numpy()
selected_top_count = max(1, int(np.ceil(len(actual_labels) * selected_queue_rate)))
top_five_count = max(1, int(np.ceil(len(actual_labels) * 0.05)))

for name, scores in risk_scores.items():
    scores = np.asarray(scores)
    model_column = name.lower().replace(" ", "_")
    scored_days[f"{model_column}_risk"] = scores

    selected_indices = np.argsort(scores)[-selected_top_count:]
    selected_flags = np.zeros(len(scores), dtype=int)
    selected_flags[selected_indices] = 1
    scored_days[f"{model_column}_flagged_selected_queue"] = selected_flags

    five_percent_indices = np.argsort(scores)[-top_five_count:]
    five_percent_flags = np.zeros(len(scores), dtype=int)
    five_percent_flags[five_percent_indices] = 1
    scored_days[f"{model_column}_flagged_top_5_percent"] = five_percent_flags

scores_path = output_dir / "evaluation_employee_day_scores.csv"
scored_days.to_csv(scores_path, index=False)

# Save confusion counts at the selected queue capacity.
confusion_rows = []
for name, matrix in confusion_matrices.items():
    tn, fp, fn, tp = matrix.ravel()
    confusion_rows.append({
        "model": name,
        "review_queue_rate": selected_queue_rate,
        "true_negative": tn,
        "false_positive": fp,
        "false_negative": fn,
        "true_positive": tp,
    })
confusion_path = output_dir / "confusion_matrix_counts.csv"
pd.DataFrame(confusion_rows).to_csv(confusion_path, index=False)

risk_columns = [column for column in scored_days.columns if column.endswith("_risk")]
employee_ranking = (
    scored_days.groupby("user")
    .agg({**{column: "max" for column in risk_columns}, target_column: "max"})
    .sort_values(best_risk_column, ascending=False)
    .reset_index()
)
ranking_path = output_dir / "evaluation_employee_ranking.csv"
employee_ranking.to_csv(ranking_path, index=False)

print("Saved model comparison:", comparison_path)
print("Saved employee-day scores and queue flags:", scores_path)
print("Saved selected-capacity confusion matrix counts:", confusion_path)
print("Saved employee ranking:", ranking_path)
print("Model selected by validation AP:", comparison_best_model)
print(f"Queue rate selected by validation F1: {selected_queue_rate:.1%}")
```

```
# Notebook code cell 31
baseline_rate = float(y_test.mean())

print(f"Positive-label␣baseline:␣{baseline_rate:.2%}")

if baseline_rate == 0:
    print("Lift␣cannot␣be␣calculated␣because␣the␣evaluation␣set␣has␣no␣positive␣labels.")
else:
    model_comparison["Test␣AP␣lift␣over␣baseline"] = (
        model_comparison["Test␣AP"] / baseline_rate
    )

    for _, row in model_comparison.iterrows():
        print(
            f"{row['Model']}:␣"
            f"{row['Test␣AP␣lift␣over␣baseline']:.1f}x␣baseline"
        )

    display(
        model_comparison.style.format({
            "Validation␣AP": "{:.3f}",
            "Validation␣precision␣@␣5%": "{:.3f}",
            "Validation␣recall␣@␣5%": "{:.3f}",
            "Test␣precision␣@␣5%": "{:.3f}",
            "Test␣recall␣@␣5%": "{:.3f}",
            "Test␣accuracy": "{:.3f}",
            "Test␣F1": "{:.3f}",
            "Test␣AP": "{:.3f}",
            "Test␣AP␣lift␣over␣baseline": "{:.1f}x",
        })
    )

# Notebook code cell 33
import joblib

print("Retraining␣models␣on␣all␣labeled␣employee-day␣data...")

for name, model in models.items():
    if name == "Isolation␣Forest":
        model.fit(X)
    elif name == "Random␣Forest" and selected_synthetic_augmentation[name]:
        synthetic_X = make_synthetic_positive_counts(X, y, random_state=2027)
        augmented_X = pd.concat([X.reset_index(drop=True), synthetic_X], ignore_index=True)
        augmented_y = pd.concat(
            [y.reset_index(drop=True), pd.Series(np.ones(len(synthetic_X), dtype=int))],
            ignore_index=True,
        )
        model.fit(augmented_X, augmented_y)
        print(f"Random␣Forest␣final␣fit␣includes␣{len(synthetic_X):,}␣synthetic␣training␣rows.")
    else:
        model.fit(X, y)

model_bundle = {
    "models": models,
    "feature_columns": feature_columns,
    "model_names": list(models.keys()),
    "synthetic_augmentation": {
        "method": "Poisson/binomial␣count␣perturbation␣of␣positive␣training␣rows",
        "selected_by_validation": selected_synthetic_augmentation,
```

```
    },
}

model_path = output_dir / "insider_investigation_model.joblib"
joblib.dump(model_bundle, model_path)

print("Saved all four models to:", model_path)
print("Models saved:", ", ".join(model_bundle["model_names"]))


# Notebook code cell 34
from sklearn.metrics import classification_report

print(f"\n{name}")
print(classification_report(
    actual_labels,
    predicted_labels,
    labels=[0, 1],
    target_names=["Not in incident window", "In incident window"],
    zero_division=0,
    digits=3,
))
```

10.6.2 FastAPI service source

The service listing records the local endpoints, request validation, temporary-file staging, progress reporting, scoring invocation, and output handling implemented by the API.

Listing 10.6: Complete FastAPI service source snapshot

```
"""Local API for the React interface; run with `uvicorn api:app --reload`."""

from pathlib import Path
from shutil import copyfileobj
from tempfile import TemporaryDirectory
from threading import Lock

import pandas as pd
from fastapi import FastAPI, File, HTTPException, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

from forensic_core import MODEL_NAMES, REQUIRED_FILES, friendly_activity_table, load_model_bundle,
    score_folder


PROJECT_DIR = Path(__file__).resolve().parent
MODEL_PATH = PROJECT_DIR / "sprint1_output" / "insider_investigation_model.joblib"
RESULTS_DIR = PROJECT_DIR / "sprint1_output"

app = FastAPI(title="Forensio API", docs_url=None, redoc_url=None)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)

analysis_lock = Lock()
employee_days: pd.DataFrame | None = None
ranking: pd.DataFrame | None = None
```

```
analysis_progress = {"percent": 0, "stage": "Waiting␣to␣start", "active": False}


class AnalyzeRequest(BaseModel):
    folder: str


class EmployeeRequest(BaseModel):
    user: str


def _records(frame: pd.DataFrame) -> list[dict]:
    clean = frame.astype(object).where(pd.notna(frame), None)
    return clean.to_dict(orient="records")


def _analyze_folder(folder: Path) -> dict:
    global employee_days, ranking, analysis_progress
    if not MODEL_PATH.is_file():
        raise HTTPException(status_code=503, detail="The␣four-model␣bundle␣is␣missing.␣Run␣notebook␣Cell␣31
    ␣first.")
    try:
        with analysis_lock:
            analysis_progress = {"percent": 15, "stage": "Preparing␣analysis", "active": True}
            bundle = load_model_bundle(MODEL_PATH)
            def update_progress(percent, stage):
                global analysis_progress
                analysis_progress = {
                    "percent": 15 + round(percent * 0.85),
                    "stage": stage,
                    "active": True,
                }

            days, report = score_folder(folder, bundle, progress_callback=update_progress)
            RESULTS_DIR.mkdir(exist_ok=True)
            days.to_csv(RESULTS_DIR / "dashboard_day_scores.csv", index=False)
            report.to_csv(RESULTS_DIR / "dashboard_report.csv", index=False)
            employee_days, ranking = days, report
            analysis_progress = {"percent": 100, "stage": "Analysis␣complete", "active": False}
    except HTTPException:
        raise
    except Exception as error:
        analysis_progress = {"percent": 0, "stage": "Analysis␣failed", "active": False}
        raise HTTPException(status_code=400, detail=f"Analysis␣could␣not␣be␣completed:␣{error}") from error

    suspicious = report[report["status"] == "SUSPICIOUS"]
    return {
        "employees": _records(report),
        "summary": {
            "employees": int(len(report)),
            "flagged": int(len(suspicious)),
            "review": int(len(report) - len(suspicious)),
            "activeDays": int(days["day"].nunique()),
        },
        "models": MODEL_NAMES,
    }


@app.get("/api/health")
def health():
    return {
```

```
        "ready": MODEL_PATH.is_file(),
        "message": "Model␣bundle␣is␣ready." if MODEL_PATH.is_file() else "Model␣bundle␣is␣missing.␣Run␣
     notebook␣Cell␣31.",
    }


@app.get("/api/progress")
def progress():
    return analysis_progress


@app.post("/api/analyze")
def analyze(request: AnalyzeRequest):
    folder = Path(request.folder).expanduser()
    if not folder.is_dir():
        raise HTTPException(status_code=400, detail="That␣folder␣could␣not␣be␣found␣on␣this␣computer.")
    return _analyze_folder(folder)


@app.post("/api/analyze-upload")
def analyze_upload(files: list[UploadFile] = File(...)):
    expected = set(REQUIRED_FILES)
    selected: dict[str, UploadFile] = {}
    optional = {"employees.csv", "employee.csv", "users.csv", "user.csv"}
    for upload in files:
        name = Path(upload.filename or "").name.lower()
        if name in expected or name in optional:
            if name in selected:
                raise HTTPException(status_code=400, detail=f"The␣selected␣folder␣contains␣duplicate␣{name}
     ␣files.")
            selected[name] = upload
    missing = sorted(expected - selected.keys())
    if missing:
        raise HTTPException(status_code=400, detail="The␣selected␣folder␣is␣missing:␣" + ",␣".join(missing)
     )

    try:
        with TemporaryDirectory(prefix="insider-risk-") as temporary:
            folder = Path(temporary)
            for name, upload in selected.items():
                with (folder / name).open("wb") as target:
                    copyfileobj(upload.file, target, length=1024 * 1024)
            return _analyze_folder(folder)
    finally:
        for upload in files:
            upload.file.close()


@app.post("/api/employee")
def employee_detail(request: EmployeeRequest):
    if ranking is None or employee_days is None:
        raise HTTPException(status_code=409, detail="Run␣an␣analysis␣before␣opening␣employee␣details.")
    selected = ranking[ranking["user"].astype(str) == request.user]
    if selected.empty:
        raise HTTPException(status_code=404, detail="Employee␣was␣not␣found␣in␣this␣report.")
    report_row = selected.iloc[0]
    days = employee_days[employee_days["user"].astype(str) == request.user]
    activity = friendly_activity_table(days)
    score_names = {
        "isolation_forest_rank": "Isolation␣Forest",
        "logistic_regression_rank": "Logistic␣Regression",
```

```
        "decision_tree_rank": "Decision Tree",
        "random_forest_rank": "Random Forest",
    }
    scores = [
        {"model": label, "score": round(float(report_row[column]) * 100, 1)}
        for column, label in score_names.items()
        if column in report_row
    ]
    name = report_row.get("name")
    return {
        "user": request.user,
        "name": None if pd.isna(name) else str(name),
        "status": str(report_row["status"]),
        "whyFlagged": str(report_row["why_flagged"]),
        "modelAgreement": int(report_row["model_agreement"]),
        "activeDays": int(report_row["active_days"]),
        "scores": scores,
        "activity": _records(activity),
    }
```

10.6.3 Feature processing and ranking source

The core module listing documents the shared CSV processing, feature alignment, model scoring, percentile ranks, and employee-level consensus logic used by the dashboard.

Listing 10.7: Complete feature processing and ranking source snapshot

```
from pathlib import Path
import subprocess

import joblib
import numpy as np
import pandas as pd


REQUIRED_FILES = ["logon.csv", "device.csv", "file.csv", "email.csv", "http.csv"]
MODEL_NAMES = [
    "Isolation Forest",
    "Logistic Regression",
    "Decision Tree",
    "Random Forest",
]
CHUNK_SIZE = 25_000


def choose_data_folder():
    """Show a macOS folder picker. Data stays in the selected local folder."""
    script = (
        'POSIX path of (choose folder with prompt '
        '"Choose the folder containing the five activity CSV files")'
    )
    result = subprocess.run(
        ["osascript", "-e", script],
        capture_output=True,
        text=True,
        check=False,
    )
    if result.returncode != 0:
        return None
```

```
    selected = result.stdout.strip()
    return Path(selected) if selected else None


def load_model_bundle(model_path):
    """Load␣and␣check␣the␣bundle␣created␣by␣notebook␣Cell␣31."""
    bundle = joblib.load(model_path)
    if not isinstance(bundle, dict) or "models" not in bundle:
        raise ValueError(
            "This␣model␣file␣does␣not␣contain␣all␣four␣models.␣"
            "Run␣notebook␣Cell␣31,␣then␣restart␣the␣dashboard."
        )
    missing_models = [name for name in MODEL_NAMES if name not in bundle["models"]]
    if missing_models:
        raise ValueError("Model␣file␣is␣missing:␣" + ",␣".join(missing_models))
    if not bundle.get("feature_columns"):
        raise ValueError("The␣model␣file␣is␣missing␣its␣feature␣column␣list.")
    return bundle


def process_activity_file(file_path):
    """Aggregate␣one␣source␣log␣into␣the␣same␣employee-day␣features␣as␣the␣notebook."""
    file_path = Path(file_path)
    source = file_path.stem.lower()
    headers = [str(c).strip().lower() for c in pd.read_csv(file_path, nrows=0).columns]
    useful = ["date", "user", "activity", "attachments"]
    selected = [column for column in useful if column in headers]

    if "date" not in selected or "user" not in selected:
        raise ValueError(f"{file_path.name}␣must␣contain␣'date'␣and␣'user'␣columns.")

    daily = None
    for chunk in pd.read_csv(
        file_path,
        usecols=lambda c: str(c).strip().lower() in selected,
        chunksize=CHUNK_SIZE,
        low_memory=False,
    ):
        chunk.columns = [str(c).strip().lower() for c in chunk.columns]
        chunk["user"] = chunk["user"].astype("string").str.strip()
        dates = pd.to_datetime(chunk["date"], errors="coerce", format="mixed")
        missing_user = chunk["user"].isna() | chunk["user"].eq("")
        keep = ~(missing_user | dates.isna() | chunk.duplicated())
        chunk = chunk.loc[keep].copy()
        dates = dates.loc[keep]
        if chunk.empty:
            continue

        chunk["day"] = dates.dt.strftime("%Y-%m-%d").to_numpy()
        hour = dates.dt.hour
        chunk["after_hours_count"] = ((hour < 6) | (hour >= 18)).astype("int8").to_numpy()
        chunk["event_count"] = 1
        columns = ["event_count", "after_hours_count"]

        if source == "device" and "activity" in chunk:
            chunk["device_connect_count"] = (
                chunk["activity"].astype("string").str.lower().eq("connect").astype("int8")
            )
            columns.append("device_connect_count")

        if source == "email" and "attachments" in chunk:
```

```
            chunk["attachment_count"] = (
                pd.to_numeric(chunk["attachments"], errors="coerce").fillna(0)
            )
            columns.append("attachment_count")

        part = chunk.groupby(["user", "day"])[columns].sum()
        part = part.rename(columns={c: f"{source}_{c}" for c in columns})
        daily = part if daily is None else daily.add(part, fill_value=0)

    if daily is None:
        return pd.DataFrame()
    return daily


def _load_optional_names(data_folder):
    """Read names only when a small, separate employee lookup file is present."""
    candidates = ["employees.csv", "employee.csv", "users.csv", "user.csv"]
    for filename in candidates:
        path = Path(data_folder) / filename
        if not path.is_file():
            continue
        lookup = pd.read_csv(path, low_memory=False)
        normalized = {str(c).strip().lower(): c for c in lookup.columns}
        user_col = next(
            (normalized[c] for c in ("user", "employee", "employee_id", "user_id") if c in normalized),
            None,
        )
        name_col = next(
            (normalized[c] for c in ("name", "employee_name", "full_name") if c in normalized),
            None,
        )
        if user_col and name_col:
            result = lookup[[user_col, name_col]].copy()
            result.columns = ["user", "name"]
            result["user"] = result["user"].astype("string").str.strip()
            result["name"] = result["name"].astype("string").str.strip()
            return result.dropna(subset=["user"]).drop_duplicates("user")
    return None


def _score(model, model_name, values):
    """Return larger values for higher risk or more unusual activity."""
    if model_name == "Isolation Forest":
        return -np.asarray(model.decision_function(values), dtype=float)
    if hasattr(model, "predict_proba"):
        return np.asarray(model.predict_proba(values)[:, 1], dtype=float)
    return np.asarray(model.decision_function(values), dtype=float)


def score_folder(data_folder, bundle, progress_callback=None):
    """Build employee-day features, score with all four models, and rank employees."""
    def report_progress(percent, message):
        if progress_callback is not None:
            progress_callback(percent, message)

    data_folder = Path(data_folder)
    missing = [name for name in REQUIRED_FILES if not (data_folder / name).is_file()]
    if missing:
        raise FileNotFoundError("Selected folder is missing: " + ", ".join(missing))

    sources = []
```

```
    for index, name in enumerate(REQUIRED_FILES):
        report_progress(index * 12, f"Reading␣and␣aggregating␣{name}␣({index␣+␣1}␣of␣{len(REQUIRED_FILES)})
     ")
        sources.append(process_activity_file(data_folder / name))
        report_progress((index + 1) * 12, f"Finished␣{name}␣({index␣+␣1}␣of␣{len(REQUIRED_FILES)})")
    sources = [frame for frame in sources if not frame.empty]
    if not sources:
        raise ValueError("The␣selected␣activity␣files␣contained␣no␣usable␣records.")

    employee_days = pd.concat(sources, axis=1).fillna(0)
    employee_days = employee_days.groupby(level=["user", "day"]).sum().reset_index()
    report_progress(65, "Preparing␣employee-day␣features")

    feature_columns = bundle["feature_columns"]
    for column in feature_columns:
        if column not in employee_days.columns:
            employee_days[column] = 0
    model_input = employee_days[feature_columns].apply(pd.to_numeric, errors="coerce").fillna(0)

    score_columns = []
    for index, name in enumerate(MODEL_NAMES):
        scores = _score(bundle["models"][name], name, model_input)
        # Percentiles put the four different score scales on a comparable rank scale.
        column = name.lower().replace("␣", "_") + "_rank"
        employee_days[column] = pd.Series(scores).rank(pct=True, method="average").to_numpy()
        score_columns.append(column)
        report_progress(65 + (index + 1) * 6, f"Scoring␣with␣{name}␣({index␣+␣1}␣of␣{len(MODEL_NAMES)})")

    employee_days["consensus_score"] = employee_days[score_columns].mean(axis=1)

    # Rank each employee by their highest-risk day for each model.
    aggregations = {
        "consensus_score": ("consensus_score", "max"),
        "active_days": ("day", "nunique"),
    }
    aggregations.update({column: (column, "max") for column in score_columns})
    ranking = employee_days.groupby("user").agg(**aggregations).reset_index()

    # This is a review-priority rule, not a probability or proof of wrongdoing.
    for column in score_columns:
        ranking[column + "_top_10"] = ranking[column].rank(pct=True, method="average") >= 0.90
    agreement_columns = [column + "_top_10" for column in score_columns]
    ranking["model_agreement"] = ranking[agreement_columns].sum(axis=1).astype(int)
    queue_size = max(1, int(np.ceil(len(ranking) * 0.05)))
    consensus_cutoff = ranking["consensus_score"].nlargest(queue_size).min()
    ranking["status"] = np.where(
        (ranking["consensus_score"] >= consensus_cutoff)
        | (ranking["model_agreement"] >= 2),
        "SUSPICIOUS",
        "REVIEW",
    )

    # Summarize activity signals on each employee's highest-consensus day.
    signals = [
        ("After-hours␣activity", "logon_after_hours_count"),
        ("USB␣connections", "device_device_connect_count"),
        ("File␣activity", "file_event_count"),
        ("Email␣activity", "email_event_count"),
        ("Web␣activity", "http_event_count"),
    ]
    reasons = {}
```

```
    for user, group in employee_days.groupby("user", sort=False):
        row = group.loc[group["consensus_score"].idxmax()]
        unusual = []
        for label, column in signals:
            if column not in employee_days.columns:
                continue
            value = float(row[column])
            cutoff = float(employee_days[column].quantile(0.95))
            if value > 0 and cutoff > 0 and value >= cutoff:
                unusual.append(f"{label} ({value:g} events on {row['day']})")
        reasons[user] = "; ".join(unusual[:3]) or (
            f"Activity ranked highly across the four models on {row['day']}"
        )

    ranking["why_flagged"] = ranking["user"].map(reasons)
    report_progress(92, "Building investigation report")
    names = _load_optional_names(data_folder)
    if names is not None:
        ranking = ranking.merge(names, on="user", how="left")
        ranking["name"] = ranking["name"].replace("", pd.NA)
    else:
        ranking["name"] = pd.NA

    ranking["_status_order"] = ranking["status"].map({"SUSPICIOUS": 0, "REVIEW": 1})
    ranking = ranking.sort_values(
        ["_status_order", "consensus_score"], ascending=[True, False]
    ).drop(columns="_status_order").reset_index(drop=True)

    employee_days = employee_days.merge(
        ranking[["user", "name", "status"]], on="user", how="left"
    )
    report_progress(100, "Analysis complete")
    return employee_days, ranking


def friendly_activity_table(user_days):
    labels = {
        "day": "Date",
        "consensus_score": "Combined rank",
        "logon_event_count": "Logins",
        "logon_after_hours_count": "After-hours",
        "device_device_connect_count": "USB connects",
        "file_event_count": "Files",
        "email_event_count": "Emails",
        "http_event_count": "Web visits",
    }
    columns = [column for column in labels if column in user_days.columns]
    return (
        user_days[columns]
        .sort_values("consensus_score", ascending=False)
        .head(20)
        .rename(columns=labels)
    )
```

10.6.4 Alternative Streamlit interface source

The project also retains an alternative Streamlit interface. Its source is included here for completeness; the primary demonstration described in the main chapters uses React with FastAPI.

Listing 10.8: Complete alternative Streamlit application source snapshot

```
# LaTeX-safe listing derived from app.py; Unicode string literals use equivalent Python escapes.
from pathlib import Path

import pandas as pd
import streamlit as st

from forensic_core import (
    choose_data_folder,
    friendly_activity_table,
    load_model_bundle,
    score_folder,
)

PROJECT_DIR = Path(__file__).resolve().parent
MODEL_PATH = PROJECT_DIR / "sprint1_output" / "insider_investigation_model.joblib"
RESULTS_DIR = PROJECT_DIR / "sprint1_output"

st.set_page_config(page_title="Forensio", page_icon="\U0001F50E", layout="wide")
st.markdown("""
<style>
.stApp␣{␣background:␣#f3f6fa;␣color:␣#202a35;␣}
[data-testid="stSidebar"]␣{␣background:␣#10243b;␣}
[data-testid="stSidebar"]␣*␣{␣color:␣#f0f4f8␣!important;␣}
.block-container␣{␣max-width:␣1380px;␣padding-top:␣1.8rem;␣}
h1,␣h2,␣h3␣{␣color:␣#172330;␣letter-spacing:␣-.02em;␣}
[data-testid="stMetric"]␣{␣background:␣#ffffff;␣border:␣1px␣solid␣#e2e9f0;
␣␣border-radius:␣14px;␣padding:␣1rem;␣box-shadow:␣0␣3px␣12px␣#1720330c;␣}
[data-testid="stMetricValue"]␣{␣color:␣#315f91;␣}
.stButton␣>␣button[kind="primary"]␣{␣background:␣#396a9e;␣border:␣0;
␣␣border-radius:␣9px;␣font-weight:␣650;␣}
[data-testid="stDataFrame"]␣{␣border:␣1px␣solid␣#e2e9f0;␣border-radius:␣10px;␣}
.report-card␣{␣background:␣#ffffff;␣border:␣1px␣solid␣#e2e9f0;
␣␣border-left:␣5px␣solid␣#4779ad;␣border-radius:␣12px;␣padding:␣1rem␣1.2rem;
␣␣margin:␣.65rem␣0;␣}
.report-status␣{␣color:␣#315f91;␣font-weight:␣750;␣letter-spacing:␣.04em;␣}
.analysis-animation␣{␣display:␣flex;␣align-items:␣center;␣gap:␣.7rem;␣color:␣#315f91;
␣␣font-weight:␣650;␣margin:␣.35rem␣0␣.75rem;␣}
.analysis-pulse␣{␣display:␣inline-flex;␣gap:␣4px;␣}
.analysis-pulse␣span␣{␣width:␣7px;␣height:␣7px;␣border-radius:␣50%;␣background:␣#4779ad;
␣␣animation:␣analysis-bounce␣1s␣infinite␣ease-in-out;␣}
.analysis-pulse␣span:nth-child(2)␣{␣animation-delay:␣.15s;␣}
.analysis-pulse␣span:nth-child(3)␣{␣animation-delay:␣.3s;␣}
.analysis-track␣{␣height:␣4px;␣overflow:␣hidden;␣border-radius:␣4px;␣background:␣#dce7f1;
␣␣margin:␣.45rem␣0␣.85rem;␣}
.analysis-track::after␣{␣content:␣"";␣display:␣block;␣width:␣38%;␣height:␣100%;
␣␣background:␣#4779ad;␣border-radius:␣inherit;␣animation:␣analysis-sweep␣1.25s␣infinite␣ease-in-out;␣}
@keyframes␣analysis-bounce␣{␣0%,␣60%,␣100%␣{␣transform:␣translateY(0);␣opacity:␣.55;␣}
␣␣30%␣{␣transform:␣translateY(-4px);␣opacity:␣1;␣}␣}
@keyframes␣analysis-sweep␣{␣from␣{␣transform:␣translateX(-110%);␣}
␣␣to␣{␣transform:␣translateX(290%);␣}␣}
</style>
""", unsafe_allow_html=True)

st.title("Forensio")
st.caption("Four-model␣results␣prioritize␣investigation.␣A␣flag␣is␣not␣proof␣of␣wrongdoing.")
st.sidebar.markdown("###␣\U0001F50E␣Forensio")
st.sidebar.caption("Decision␣Tree␣\u00B7␣Random␣Forest␣\u00B7␣Logistic␣Regression␣\u00B7␣Isolation␣Forest")

if not MODEL_PATH.is_file():
    st.error("The␣four-model␣file␣is␣missing.␣Run␣notebook␣Cell␣31,␣then␣restart␣this␣app.")
```

```
    st.stop()

try:
    bundle = load_model_bundle(MODEL_PATH)
except Exception as error:
    st.error(str(error))
    st.stop()

if "analysis_done" not in st.session_state:
    st.session_state.analysis_done = False

st.subheader("Choose activity folder")
st.write("Choose the folder containing logon.csv, device.csv, file.csv, email.csv, and http.csv.")

if st.button("Choose folder", type="primary"):
    folder = choose_data_folder()
    if folder:
        st.session_state.data_folder = str(folder)
    else:
        st.info("No folder selected.")

folder_text = st.session_state.get("data_folder")
if folder_text:
    folder = Path(folder_text)
    st.success(f"Selected folder: {folder.name}")
    if st.button("Analyze folder", type="primary"):
        try:
            animation_slot = st.empty()
            animation_slot.markdown(
                '<div class="analysis-animation">Analyzing activity data'
                '<span class="analysis-pulse"><span></span><span></span><span></span></span></div>'
                '<div class="analysis-track"></div>',
                unsafe_allow_html=True,
            )
            progress_bar = st.progress(0, text="Starting analysis")
            with st.spinner("Analyzing activity data"):
                days, report = score_folder(
                    folder,
                    bundle,
                    progress_callback=lambda percent, message: progress_bar.progress(
                        percent, text=message
                    ),
                )
            st.session_state.employee_days = days
            st.session_state.ranking = report
            st.session_state.analysis_done = True
            RESULTS_DIR.mkdir(exist_ok=True)
            days.to_csv(RESULTS_DIR / "dashboard_day_scores.csv", index=False)
            report.to_csv(RESULTS_DIR / "dashboard_report.csv", index=False)
            progress_bar.progress(100, text="Analysis complete")
            animation_slot.empty()
            st.success("Analysis complete. The report files were saved in sprint1_output.")
        except Exception as error:
            if "animation_slot" in locals():
                animation_slot.empty()
            st.error(f"Analysis could not be completed: {error}")

if st.session_state.analysis_done:
    ranking = st.session_state.ranking
    employee_days = st.session_state.employee_days
    flagged = ranking[ranking["status"] == "SUSPICIOUS"].copy()
```

```
    st.divider()
    st.header("Investigation Report")
    st.caption(
        "The combined score is a relative rank, not a probability. SUSPICIOUS "
        "means prioritized for human review based on the four-model review rule."
    )

    top_ten = flagged.head(10)
    if top_ten.empty:
        st.info("No employee crossed the review rule for this dataset.")
    else:
        st.markdown("### Top 10 employees flagged for review")
        for _, row in top_ten.iterrows():
            employee_name = row.get("name")
            name_line = (
                f"<p><b>Name:</b> {employee_name}</p>"
                if pd.notna(employee_name) and str(employee_name).strip()
                else ""
            )
            st.markdown(
                f"""
                <div class="report-card">
                <div class="report-status">Status: SUSPICIOUS</div>
                <p><b>Employee:</b> {row['user']}</p>
                {name_line}
                <p><b>Flagged for:</b> {row['why_flagged']}</p>
                <p><b>Model agreement:</b> {int(row['model_agreement'])} / 4</p>
                </div>
                """,
                unsafe_allow_html=True,
            )

    st.markdown("### Other employees")
    top_ten_ids = set(top_ten["user"].tolist())
    table = ranking[~ranking["user"].isin(top_ten_ids)].head(25).copy()
    table["Combined rank"] = (table["consensus_score"] * 100).round(1)
    table["Model agreement"] = table["model_agreement"].astype(str) + " / 4"
    table = table.rename(columns={
        "user": "Employee",
        "name": "Name",
        "status": "Status",
        "why_flagged": "Flagged for",
        "active_days": "Active days",
    })
    shown = [
        column for column in
        ["Status", "Employee", "Name", "Combined rank", "Model agreement", "Active days", "Flagged for"]
        if column in table.columns
    ]
    if table.empty:
        st.caption("There are no additional employees to list.")
    else:
        st.dataframe(table[shown], hide_index=True, use_container_width=True)

    selected = st.selectbox("Review employee activity", ranking["user"].tolist())
    selected_days = employee_days[employee_days["user"] == selected].copy()
    selected_report = ranking.loc[ranking["user"] == selected].iloc[0]
    st.subheader(f"Activity details: {selected}")
    st.write(f"**Status:** {selected_report['status']}")
    st.write(f"**Flagged for:** {selected_report['why_flagged']}")
```

```
    if pd.notna(selected_report["name"]) and str(selected_report["name"]).strip():
        st.write(f"**Name:** {selected_report['name']}")

    model_columns = [
        "isolation_forest_rank",
        "logistic_regression_rank",
        "decision_tree_rank",
        "random_forest_rank",
    ]
    model_scores = selected_report[model_columns].rename({
        "isolation_forest_rank": "Isolation Forest",
        "logistic_regression_rank": "Logistic Regression",
        "decision_tree_rank": "Decision Tree",
        "random_forest_rank": "Random Forest",
    }).to_frame("Rank score")
    model_scores["Rank score"] = (model_scores["Rank score"] * 100).round(1)
    st.markdown("**Scores from each model**")
    st.dataframe(model_scores, use_container_width=True)

    st.markdown("**Highest-ranked activity days**")
    st.dataframe(
        friendly_activity_table(selected_days),
        hide_index=True,
        use_container_width=True,
    )

st.caption(
    "Model scores are relative ranks within the selected data. Confirm findings "
    "against original logs and other evidence before drawing conclusions."
)
```

10.7 Supplementary Project Record

This section gathers additional implementation details that are useful when reading, reproducing, or demonstrating the project. It follows the same report class, page dimensions, typography, line spacing, headers, and numbering used throughout this report. It documents the implemented prototype and identifies proposed checks separately from completed work.

10.7.1 Two Distinct Project Workflows

The project has an offline model-development workflow and a separate interactive scoring workflow. They share feature definitions, but they have different inputs, objectives, and outputs. Keeping them distinct is necessary to interpret the reported test metrics correctly.

10.7.1.1 Offline development workflow

The notebook reads the synthetic activity files and answer file, constructs employee-day features, attaches known-incident-window labels, and creates employee-grouped train, validation, and test partitions. Candidate estimators are fitted on training data. Validation Average Precision is used to compare ranking performance, and the candidate review capacities are compared on validation data. The held- out test partition is then used for the reported final estimates.

After the evaluation stage, the notebook retrains the artifact models using the available labelled employee-day data. For the Random Forest, the selected synthetic augmentation rule is applied during this final fit when validation selected that variant. This full-data model bundle is

intended for interactive demonstration. The test metrics describe the earlier held-out evaluation model and must not be presented as a fresh evaluation of the final bundle, since that bundle has seen the records used in the test partition during its final fit.

10.7.1.2 Interactive scoring workflow

The local dashboard accepts activity files without the answer file. It rebuilds employee-day features, aligns the resulting columns to the feature order saved in the model bundle, and scores every row with all four fitted estimators. It converts each score series to a percentile rank within the selected folder, computes a consensus value, aggregates daily ranks to employee rankings, and writes two CSV summaries. Since this inference workflow does not receive answer labels, it cannot calculate precision, recall, or incident-window correctness for the selected folder.

The distinction is summarized in Table 10.2.

Table 10.2: Offline evaluation and interactive scoring boundaries

Aspect	Offline development and evaluation	Interactive dashboard scoring
Activity input	CERT r4.2 logs	User-selected folder with five activity logs
Answer file	Used to construct known-window labels	Not required and not uploaded
Purpose	Compare models and estimate ranking metrics on held-out groups	Generate a relative review queue for a selected folder
Model fit	Train, validate, evaluate, then create a final demonstration bundle	Load saved estimators; no fitting occurs during a folder analysis
Evaluation target	Employee-day known-window label	No correctness target is available at inference time
Main output	Saved comparison metrics and test predictions	Employee ranking and employee-day score CSV files
Interpretation	Research estimate for a fixed split and label rule	Within-folder prioritization; not a measured probability

10.7.2 Detailed Input Contract

The current shared scoring implementation expects five filenames and uses normalized column names. The fields in Table 10.3 describe the implemented feature processor. Optional columns affect only the feature derived from that column; the basic event count still uses each retained row. A missing required date or user field stops processing for that source.

Table 10.3: Input columns and derived values in the dashboard processor

Source	Required columns	Optional columns	Derived employee-day values
Logon	`date`, `user`	None used by the feature processor	Event count and after-hours event count
Device	`date`, `user`	`activity`	Event count and after-hours count; connection count when activity is “connect”
File	`date`, `user`	None used by the feature processor	Event count and after-hours event count

Source	Required columns	Optional columns	Derived employee-day values
Email	`date`, `user`	`attachments`	Event count and after-hours count; numeric attachment sum when available
HTTP	`date`, `user`	None used by the feature processor	Event count and after-hours event count
Optional employee lookup	A recognized user identifier column and name column	Lookup filename may be one of the supported variants	Display name attached to the employee report; no model feature is created

The timestamp field is parsed with invalid values coerced to missing. Rows with a missing or blank user, an invalid date, or a duplicate within the current chunk do not contribute to the aggregate. User identifiers are trimmed as strings. Date values are converted to a day key, while the hour is used only to create the fixed after-hours indicator. These rules mean that the aggregate is not a lossless representation of the input: the original event order and the full timestamp are not retained as model inputs.

10.7.2.1 Feature alignment

The saved model bundle carries an ordered feature list. Before scoring, the processor checks each expected feature and inserts a zero column when that feature is absent from the selected aggregate. It then selects the ordered columns and coerces values to numeric form, filling non-numeric values with zero. This makes inference tolerant of optional source fields, but it can also hide a mismatch between the training schema and an incomplete source export. A stronger future implementation could distinguish an intentionally absent optional field from a failed or incomplete export and report both conditions explicitly.

10.7.3 Review-Queue Calculation

Let $s_{u,d,m}$ denote the raw score for user u, day d, and model m. Each model's scores are ranked within the current analysis and expressed as percentile values $r_{u,d,m}$. The daily consensus is

$$c_{u,d} = \frac{1}{4} \sum_{m=1}^{4} r_{u,d,m}.$$

The employee ranking retains the maximum value across that user's days,

$$C_u = \max_d c_{u,d},$$

and also retains each model's maximum daily percentile. Model agreement counts how many of those employee-level model ranks fall in that model's top decile. In implementation, the display status is assigned when the consensus reaches the top-five-percent employee cutoff or when at least two model ranks meet the top-decile condition. The report uses "SUSPICIOUS" for the stored status value, while the interface uses review-oriented wording such as "Prioritize"; neither string is a determination of misconduct.

10.7.3.1 Signal text and its limitations

The generated reason string is based on the employee's highest- consensus day. The code checks a limited set of counts against the 95th percentile of the corresponding feature across all employee-days in the current folder. A signal is included only when the feature is present, the value is positive, and the feature percentile cutoff is positive. At most three signal phrases are retained; if no phrase meets the rule, a generic high-ranking activity message is used.

This is a descriptive summary rather than a model explanation. The signal text does not show feature contributions, causal relationships, or the sequence of raw events. The feature quantiles are recalculated for each analysis, so a phrase may appear or disappear when the composition of the selected folder changes. Analysts should use it as a navigation aid to the aggregate day, then consult authorized source records and context through a separate process.

10.7.4 Illustrative Review Walkthrough

This walkthrough describes how an analyst could navigate a result in the demonstration. It is a procedural example and does not describe an actual employee or a saved investigation outcome.

1. The analyst starts the local API and React client and confirms that the health indicator reports the model bundle as available.
2. The analyst selects a folder containing the five named source files. The browser reports any missing required filenames before enabling analysis.
3. During submission, the interface reports upload progress and then backend processing milestones. The remaining-time text is treated as an estimate because the processing stages vary in duration.
4. After a successful response, the analyst reviews the overall number of users, flagged count, and active-day count. These are descriptive totals for the selected folder, not quality measures for the model.
5. The analyst inspects the highest-ranked employee cards and considers the four-model agreement count alongside the consensus rank. Disagreement can still be informative because the four models use different scoring principles.
6. The analyst opens an employee profile and checks the four within-folder percentile ranks, the generated signal text, and the highest-ranked days. No raw event content is shown in this profile.
7. If a lead requires further attention, the analyst checks the original authorized source records and relevant schedule or operational context, then records the outcome in an approved system outside this prototype.

At each step, the user should remember that rankings depend on the selected folder. If the folder omits dates, users, or one source file, the resulting ranks describe only the records that were successfully processed. A high rank does not establish intent, and a low rank does not establish that no incident occurred.

10.7.5 Proposed Acceptance and Quality Plan

The project report records notebook and saved-artifact checks, but it does not claim a completed end-to-end user acceptance test. The cases below form a concrete plan for a later verification run. Every case is marked as proposed so it is not confused with completed verification.

Table 10.4: Supplementary acceptance and quality plan

ID	Area	Test procedure and expected result	Status
QA01	Required-file validation	Omit one required source and verify the API names the missing file and does not score a partial set	Proposed
QA02	Header normalization	Vary header capitalization and surrounding spaces; verify normalized required headers are recognized	Proposed
QA03	Invalid timestamp	Include an invalid date and verify the row does not affect daily event counts	Proposed
QA04	Blank identity	Include an empty and whitespace-only user; verify both rows are excluded	Proposed
QA05	Chunk boundary	Duplicate a row at the end of one chunk and start of the next; document that current chunk-local deduplication may retain both	Proposed
QA06	Optional device field	Remove the activity column and verify that device event counts remain available while connection counts are absent and aligned appropriately	Proposed
QA07	Optional attachment field	Remove or corrupt the attachment column and verify email event counts still process without a fabricated attachment total	Proposed
QA08	Empty source	Provide a valid header and no data rows; verify the overall analysis handles an empty source alongside nonempty sources	Proposed
QA09	Empty selection	Provide five empty files and verify a clear no-usable-records response	Proposed
QA10	Bundle mismatch	Remove one model or the feature list from a controlled test bundle and verify loading stops with a useful diagnostic	Proposed

ID	Area	Test procedure and expected result	Status
QA11	Rank ties	Supply identical feature rows and inspect stable tie handling and resulting percentile values	Proposed
QA12	Employee detail	Open a valid employee, then an unknown identifier; verify detail and not-found behaviors respectively	Proposed
QA13	Output schema	Confirm employee-day and employee-level CSV columns, row counts, and user/day keys after one successful run	Proposed
QA14	Retention and permissions	Inspect temporary staging cleanup and generated-output permissions after success and after a forced error	Proposed
QA15	Accessibility	Complete keyboard-only navigation and screen-reader review of folder selection, progress, errors, queue, and profile	Proposed

The quality plan includes expected behavior that can be observed or recalculated, rather than relying on a subjective impression that the interface appears to work. For each completed run, a future test record should include the input fixture version, software versions, expected and observed result, evidence location, and reviewer. Sensitive real activity should not be used as a test fixture unless it has been explicitly approved and protected.

10.7.6 Artifact and Run Record

For a reproducible demonstration, a run record can be maintained alongside the generated files. Table 10.5 provides a blank schema for such a record. No values are supplied here because the report does not contain a documented end-to-end dashboard analysis run.

This record schema is a proposal, not a feature of the present API. Adding run identifiers or audit metadata would require coordinated changes to the service, UI, storage permissions, and retention process. It should be implemented only as part of a defined data-governance design.

10.7.7 Known Technical Boundaries

The current demonstration has several boundaries that should remain visible when its outputs are presented:

- The answer-file window is a proxy label and may not identify which event on a marked day relates to the scenario.
- The employee-grouped split tests unseen synthetic users, but it does not establish time-forward generalization.
- Synthetic positive augmentation is selected on one validation split and is not evidence that the generated rows match real activity patterns.

Table 10.5: Suggested run-record fields for a future demonstration

Field	Purpose
Run identifier	Distinguishes one analysis from another without using a person identifier
Input snapshot reference	Identifies approved file set using controlled location and checksums
Operator and authorization reference	Records who performed the run and its approved purpose in an authorized record system
Model bundle version	Connects the run to the estimator and ordered feature list used
Processing start/end time	Supports operational timing and resource measurement
Quality summary	Records input rows, excluded rows, available fields, and any processing warnings
Output references	Identifies generated employee and day summaries under appropriate access control
Retention and deletion date	Records how long the run artifacts may be retained and when they are removed
Review outcome reference	Links to an approved external review record without storing a sensitive narrative in the model output

- Employee-day classification metrics do not measure the employee-level dashboard rule or analyst outcomes.
- Percentile scores and queue thresholds are recalculated within each selected dataset and cannot be compared as absolute risk probabilities across runs.
- The application does not authenticate analysts, retain case decisions, or enforce authorization, access, and deletion policies.
- The captured UI images show the landing page and scoring guide; they do not demonstrate a completed end-to-end analysis screen.

These constraints define the appropriate scope of the project record: an educational, local prototype for feature engineering, comparative model evaluation, and review-queue interaction. The contents of this report do not establish suitability for employment decisions or real-world monitoring.

REFERENCES

[1] Carnegie Mellon University Software Engineering Institute, *CERT Insider Threat Test Dataset*, dataset documentation and release resources. Available at: https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=508099. Accessed October 2026.

[2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001. doi: 10.1023/A:1010933404324.

[3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining*, pp. 413–422, 2008. doi: 10.1109/ICDM.2008.17.

[4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. See also scikit-learn, "Classification metrics," https://scikit-learn.org/stable/modules/model_evaluation.html. Accessed October 2026.

[5] Scikit-learn developers, "IsolationForest API reference," https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html. Accessed October 2026.

[6] The pandas development team, "pandas documentation," https://pandas.pydata.org/docs/. Accessed October 2026.

[7] Streamlit, "Streamlit documentation," https://docs.streamlit.io/. Accessed October 2026.

[8] Meta Open Source, "React documentation," https://react.dev/. Accessed October 2026.

[9] FastAPI contributors, "FastAPI documentation," https://fastapi.tiangolo.com/. Accessed October 2026.